

AUTOMATIC PRUNING OF FINE-TUNING DATASETS FOR TRANSFORMER-BASED LANGUAGE MODELS

Mohammadreza Tayaranian* **Seyyed Hasan Mozafari** **Brett H. Meyer** **James J. Clark** **Warren J. Gross**
 Department of Electrical and Computer Engineering
 McGill University
 Montreal, Canada

ABSTRACT

Transformer-based language models have shown state-of-the-art performance on a variety of natural language understanding tasks. To achieve this performance, these models are first pre-trained on general corpus and then fine-tuned on downstream tasks. Previous work studied the effect of pruning the training set of the downstream tasks on the performance of the model on its evaluation set. In this work, we propose an automatic dataset pruning method for the training set of fine-tuning tasks. Our method is based on the model’s success rate in correctly classifying each training data point. Unlike previous work which relies on user feedback to determine subset size, our method automatically extracts training subsets that are adapted for each pair of model and fine-tuning task. Our method provides multiple subsets for use in dataset pruning that navigate the trade-off between subset size and evaluation accuracy. Our largest subset, which we also refer to as the winning ticket subset, is on average $3\times$ smaller than the original training set of the fine-tuning task. Our experiments on 5 downstream tasks and 2 language models show that, on average, fine-tuning on the winning ticket subsets results in a 0.1% increase in the evaluation performance of the model.

1 INTRODUCTION

Transformer-based language models have shown state-of-the-art performance in various natural language understanding tasks (Liu et al., 2019; Raffel et al., 2020). These models are commonly used in a transfer learning setup in which they are first pre-trained on general textual data and then transferred by fine-tuning their parameters on the training set of each downstream task. The goal of fine-tuning is to maximise the model’s performance on the evaluation set¹. It relies on the existing information in the pre-trained model in addition to the information gained during fine-tuning (Durrani et al., 2022). However, different data points in the fine-tuning dataset have different contributions in achieving this goal (Katharopoulos & Fleuret, 2018).

As shown in previous work, the contribution of individual fine-tuning data points on the evaluation performance of deep neural networks is not the same (Vodrahalli et al., 2018). Based on this observation, dataset pruning methods reduce the complexity of fine-tuning by removing data points from the training set of fine-tuning tasks (Toneva et al., 2019; Sorscher et al., 2022; Paul et al., 2021). These methods try to minimize the size of the training set of the fine-tuning task while maintaining the evaluation performance of the model after fine-tuning. They measure the usefulness of each data point in maximising the evaluation performance using scoring systems as a proxy metric. The pruning is done by removing data points with the lowest score until a certain pruning percentage is reached.

Ethayarajh et al. (2022) studied the fine-tuning datasets of transformer-based language models and reported that fine-tuning tasks have different difficulties for different pre-trained models. This finding implies that if dataset pruning is applied to the fine-tuning phase, the pruned subset needs to be created based on the needs of each model and fine-tuning task. Methods of dataset pruning for the fine-tuning of transformer-based language models address this issue by computing dataset pruning scores for each task and model pair (Swayamdipta et al., 2020; Fayyaz et al., 2022). However, existing methods do not adapt the size of the pruned subset for different tasks and models. This results in using an arbitrary subset size for multiple setups. The problem with this design choice is that a subset size which is optimal for one task may be sub-optimal for another. A subset with sub-optimal size will either include too many data points and significantly drop the evaluation performance, or too few data points and miss the opportunity to create a smaller subset. For instance, as discussed in Section 4.2, fine-tuning a pre-trained model on a subset which is 27%

* Corresponding author: mohammadreza.tayaranian@mail.mcgill.ca

¹In this context, the evaluation set may refer to either the validation or the test set of the fine-tuning task.

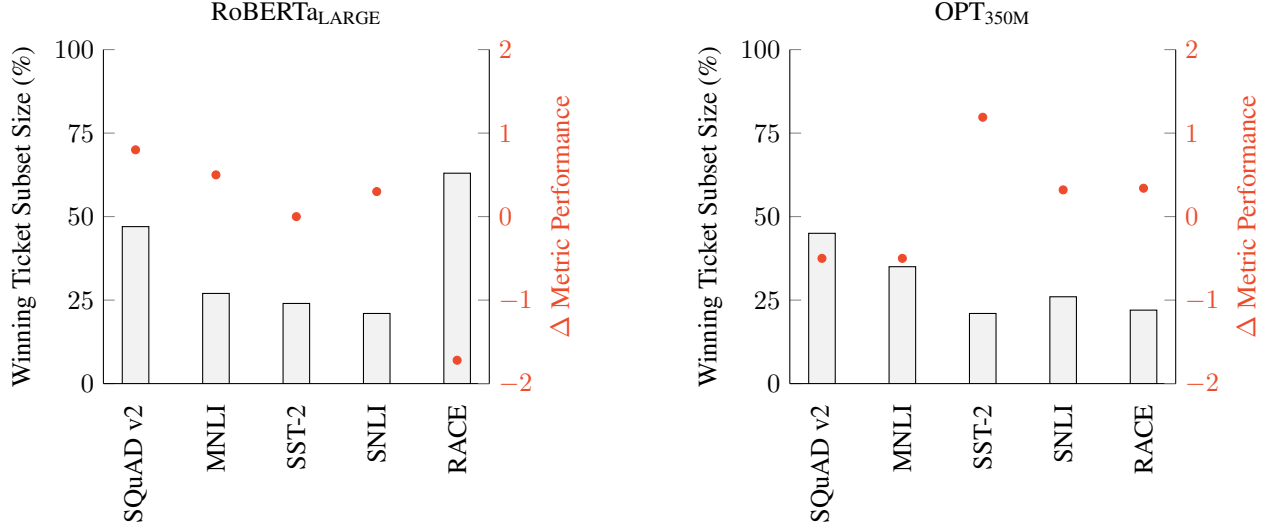


Figure 1: Results of fine-tuning two transformer-based language models, OPT_{350M} (Zhang et al., 2022) and RoBERTa_{LARGE} (Liu et al., 2019), on the winning ticket subset of various downstream tasks. For each task, the size of the winning ticket subset as a percentage of the full dataset is shown as gray bars. Δ Metric Performance, shown using red dots, is the change in the evaluation performance of the model which is fine-tuned on the full dataset, compared to the winning ticket subset. A positive Δ indicates that the winning ticket subset improved the evaluation performance of the model.

of the training set for the MNLI task results in almost no drop in the evaluation. However, using the same model and subset size for the RACE task results in more than 5% drop in the evaluation accuracy.

A simple solution is using hyperparameter search to find the optimal subset size for each task and model. This results in multiple fine-tuning runs with different sizes which is an overall costly operation considering the model size and the variety of the tasks.

In this work, we aim at pruning the training set of fine-tuning tasks for transformer-based language models. To this end, we propose a dataset scoring function, $\mathcal{H}$ -score, which is based on the output of the model during fine-tuning. The $\mathcal{H}$ -score of each data point is inversely related with its classification difficulty for the model. Across multiple runs and epochs, a data point with a score of 0 has never been correctly classified while a data point with the maximum score has always been correctly classified. The $\mathcal{H}$ -score of the i -th training data point, $\mathcal{H}_i$, is an integer which essentially distributes data points into a number of buckets.

For dataset pruning, we remove all the data points with a certain $\mathcal{H}$ -score to create training subsets from the main training set. We remove the most difficult data points as including them in the fine-tuning subset only confuses the model and destabilizes the training procedure. To further reduce the size of the pruned dataset, we also remove the least difficult data points based on the observation that their exclusion from the training set does not affect the model’s evaluation performance. Note that the list of included data points in each subset is not indicated by the user and is based on the $\mathcal{H}$ -score. Given the small size of this subset and that it helps the model achieve the same evaluation performance as the main training set, we call it the **Winning Ticket Subset**. In addition to the winning ticket subset, we create smaller subsets by removing more data point buckets with small $\mathcal{H}$ -scores, i.e. data points that are harder to learn. Overall, our proposed subsets provide a diverse set of options for the trade-off between subset size and evaluation performance where small subsets are as small as 4% of the training set of the fine-tuning task.

We conduct experiments to study the effect of fine-tuning transformer-based language models on the subsets that are created with our method. For each pair of model and task, we first fine-tune the model on the training set with 6 different seeds and use the training outputs to compute the $\mathcal{H}$ -scores. We then use the $\mathcal{H}$ -scores to create our pruned subsets and use these small subsets to fine-tune the pre-trained model. Figure 1 illustrates the change in the evaluation performance of the model when fine-tuned on the winning ticket subset compared to full fine-tuning, along with the size of this subset. On average, despite containing only 33% of the training data points, fine-tuning on these subsets results in an overall increase of 0.1% in the evaluation performance of the models. In cases like fine-tuning OPT_{350M} on the SST-2 task we even observe a 1.2% increase in the evaluation accuracy with the winning ticket subset being only

21% of the training set. As shown in Figures 4 and 5, our smaller subsets also maintain the evaluation performance. For instance, fine-tuning RoBERTa_{LARGE} on our smallest subset with 4% of training data points results in just 0.8% drop in the evaluation accuracy compared to full fine-tuning. Our dataset pruning algorithm incurs a one-time cost to achieve a significantly smaller subset. As a result, the pruned subsets can be used in future runs to reduce fine-tuning time. A prominent use-case of this method is neural architecture search (NAS) which requires thousands of GPU hours of fine-tuning on the same model and dataset to find a target architecture (Chen et al., 2020; Xu et al., 2021). Substituting the full fine-tuning dataset with our subsets can significantly reduce the search time of NAS methods while delivering a similar evaluation performance.

The contributions of this work can be summarized as:

1. We propose a dataset scoring function, $\mathcal{H}$ -score, which categorizes training data points based on the difficulty of the model in classifying them during fine-tuning.
2. Using $\mathcal{H}$ -score, we obtain smaller subsets of the training set of fine-tuning tasks. Our subsets are automatically created to minimize subset size while maximising the evaluation performance of the model fine-tuned on them. We call the subset obtained by removing the least and the most difficult data points the **Winning Ticket Subset**.
3. Our experimental results show that fine-tuning transformer-based language models on these small subsets results in an evaluation accuracy similar to using the full training set of the fine-tuning task.

2 RELATED WORK

Previous work studied the effect of pruning the training datasets commonly used for image classification tasks. Paul et al. (2021) proposed a gradient-based GraNd metric along with an estimated version, EL2N, which only uses the training loss to determine the scores of data points. Using these scores to identify the easy to learn data points of the CIFAR-10 dataset, they prune away up to 40% of the training set with minimal accuracy loss. Toneva et al. (2019) introduces the *forgetting score* which measures the rate at which a model misclassifies a data point after correctly classifying it. Unlike the *forgetting score* which removes data points that are least forgotten, our proposed $\mathcal{H}$ -score negatively rewards misclassifications across any of the fine-tuning epochs which is different from forgetting. We further study the effects of this design choice in Section 4.3. Baldock et al. (2021) measures the dataset difficulty as the number of layers required to consistently classify a data point’s label correctly. They also propose another score, *iteration learned* which is similar to the forgetting score and show that it is correlated with their other layer-based score. While the original works were designed for training randomly initialized models, Sorscher et al. (2022) prunes the training datasets for fine-tuning of pre-trained vision models. Their experiments show that, when used for fine-tuning, up to 90% of the CIFAR-10 dataset can be removed with no loss of test accuracy.

Fayyaz et al. (2022) applies the GraNd and EL2N scores to the fine-tuning tasks of transformer-based language models. Their experimental results show the suboptimal performance of these scores when applied to language models. More specifically, they show that random pruning outperforms GraNd and EL2N for subsets smaller than 70% of the training set. Moreover, pruning more than 60% of the training set using GraNd and EL2N leads to drops in the evaluation accuracy of over 20%. Swayamdipta et al. (2020) use model’s probability of the correct output to determine a dataset pruning score. Based on this score, they propose to remove the data points that are most ambiguous for the model. They also show that their ambiguous setup is more effective than other dataset pruning scores, including the *forgetting score* when applied to the fine-tuning of language models. While their idea of fine-tuning on ambiguous data points is similar to our proposed $\mathcal{H}$ -score, these methods differ both in the approach and in the achieved results. In contrast to Swayamdipta et al. (2020) which uses one fine-tuning run with 20 epochs, we use the outputs of 6 runs, each running for 3 epochs. This enables $\mathcal{H}$ -score to take the effect of the order of data points into account. Moreover, while Swayamdipta et al. (2020) uses the same pruning percentage for all tasks, our method adapts the subset size for each task and model for optimal evaluation performance. In terms of experimental results, as discussed in Section 4.2, our method creates smaller fine-tuning subsets with a smaller drop in the evaluation performance compared to the *ambiguous* setup of Swayamdipta et al. (2020).

3 METHODOLOGY

3.1 DATASET SCORING SYSTEM: $\mathcal{H}$ -SCORE

Consider the training set of a fine-tuning task, $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$. The impact of each (x_i, y_i) on the evaluation performance of the model varies based on different attributes (Vodrahalli et al., 2018; Katharopoulos & Fleuret, 2018).

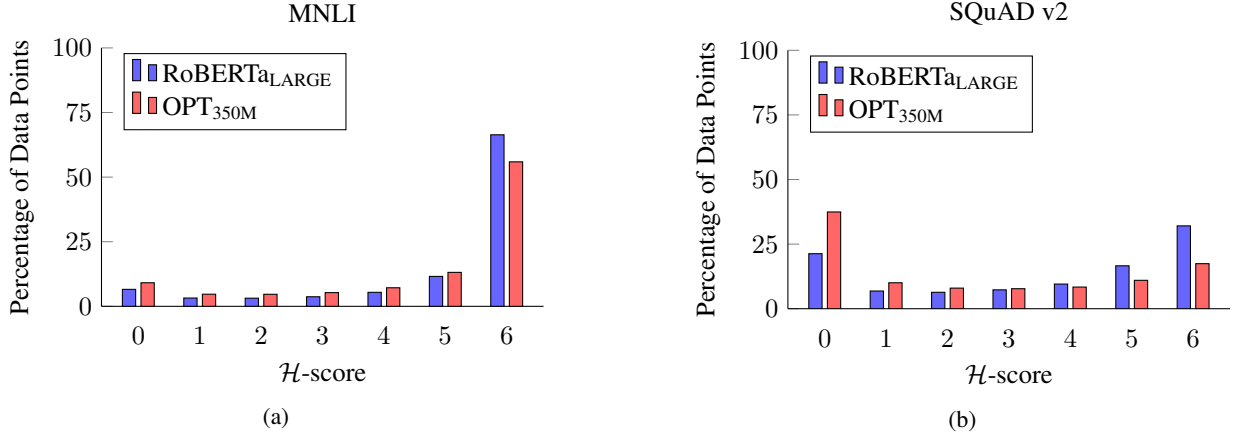


Figure 2: Distribution of the $\mathcal{H}$ -score for the training set of the MNLI and SQuAD v2 tasks, calculated using two transformer-based models RoBERTa_{LARGE} and OPT_{350M}.

We define a dataset scoring function that quantifies this impact based on the ability of the model in predicting y_i during fine-tuning.

Given a classification problem with M classes and $z_i \in \mathbb{R}^M$ as the model’s logits for x_i , the model’s prediction of the true label is calculated as $\hat{y}_i = \operatorname{argmax}_m (P(z_{im}))$. In this context, $P(z_{im})$ is model’s predicted probability for class m given by the softmax function. Fine-tuning the model for S runs with different initializations with each run taking E epochs, we store all the predictions as $\hat{Y} \in \mathbb{Z}^{N \times S \times E}$. We define our proposed dataset scoring function as $f : \hat{Y} \times Y \rightarrow \mathcal{H}$ where $\mathcal{H} \in \mathbb{Z}^N$ is the list of scores for the data points. For $i \in \{1, 2, \dots, N\}$, the $\mathcal{H}$ -score for data point (x_i, y_i) , is calculated as

$$\mathcal{H}_i = \sum_{j=1}^S \prod_{k=1}^E [\hat{Y}_{ijk} = y_i] \quad (1)$$

where $[\cdot]$ is the Iverson bracket.

For each data point, Equation 1 assigns a reward of 1 for each epoch and fine-tuning run if the model has correctly classified the true label. In the next step it multiplies this reward over all the epochs of each run to penalize data points that are misclassified at any epoch during at least one fine-tuning run. The resulting values, that are either 0 or 1, are summed for each run, resulting in $\mathcal{H}$ -scores such that $\mathcal{H}_i \in \{0, 1, \dots, S\}$.

Here we discuss the properties of a given data point, (x_i, y_i) , based on its $\mathcal{H}$ -score:

- $\mathcal{H}_i = 0$: Having a $\mathcal{H}$ -score of 0 indicates that in all of the runs this data point experienced at least one epoch where it was incorrectly classified.
- $\mathcal{H}_i = S$: This data point has been correctly classified and received a reward of 1 in all the epochs of all the fine-tuning runs which implies that correctly classifying it was relatively easier for the model.
- $\mathcal{H}_i \in \{1, 2, \dots, S-1\}$: This data point is more ambiguous for the model than previous cases as it was correctly classified in all epochs of some, but not all of the runs. More specifically, the farther the $\mathcal{H}$ -score of a data point is from 0 or S , the more ambiguous it is for the model to classify.

Figure 2 shows the distribution of the $\mathcal{H}$ -scores for the MNLI and SQuAD v2 fine-tuning datasets in a setup where $S = 6$. For the $\mathcal{H}$ -score distribution of other tasks refer to Appendix A.1.

3.2 DATASET PRUNING WITH $\mathcal{H}$ -SCORE

Given $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$ the training set of a fine-tuning task, and a set of $\mathcal{H}$ -score values, $M = \{m \mid m \in \mathbb{Z}, 0 \leq m \leq S\}$, we define a training subset $\mathcal{D}_M$ as

$$\mathcal{D}_M = \{(x_i, y_i) \mid \mathcal{H}_i \in M, 1 \leq i \leq N\}. \quad (2)$$

For instance, $\mathcal{D}_{\{0,2,5\}}$ is a subset of the training set which contains all the data points that have a $\mathcal{H}$ -score of either 0, 2, or 5.

In order to be used for dataset pruning, a training subset needs to have a smaller size than the training set. Additionally, the evaluation performance of the model that is fine-tuned on this subset needs to be close to the one fine-tuned on the full dataset. Previous work fine-tuned pre-trained models on a subset that is created by removing hard-to-learn and easy-to-learn data points from the training set (Swayamdipta et al., 2020; Ethayarajh et al., 2022). As discussed in Section 3.1, the $\mathcal{H}$ -score of a data point has an inverse relationship with its difficulty for the model. Based on these observations, we propose to use the following subsets for dataset pruning:

- $\mathcal{D}_{\{1,2,3,4,5\}}$: This subset, which we also refer to as the **Winning Ticket Subset**, is created by removing all data points (x_i, y_i) where $\mathcal{H}_i = 0$ and $\mathcal{H}_i = 6$ which correspond to the most and least difficult training data points, respectively. The former group is removed as their $\mathcal{H}$ -score of 0 means that the model is unlikely to learn them and their presence in the training subset only confuses the model. On the other hand, data points with $\mathcal{H}_i = 6$ are too easy to learn and can be avoided without damaging the evaluation performance. As shown in Figures 2 and 7, the majority of data points are associated with either of these $\mathcal{H}$ -scores.
- $\mathcal{D}_{\{2,3,4,5\}}$, $\mathcal{D}_{\{3,4,5\}}$, $\mathcal{D}_{\{4,5\}}$, and $\mathcal{D}_{\{5\}}$: These subsets enable pruning to even smaller percentages than the winning ticket subset. To create these subsets we start from the winning ticket subset and each time remove the data points with the smallest $\mathcal{H}$ -score. Although the removed data points are not as difficult as those with $\mathcal{H}_i = 0$, they are the most difficult ones in each subset.
- $\mathcal{D}_{\{4\}}$ and $\mathcal{D}_{\{2,3,4\}}$: As discussed in Section 3.1, data points that have $\mathcal{H}$ -scores that are farthest from 0 and 6 are the most ambiguous for the model. These two subsets are also dominated by data points with middle scores and thus represent the most ambiguous subsets.

Although the data points of each subset are determined based on the $\mathcal{H}$ -score, the variety of subsets helps the user decide on the trade-off between subset size and evaluation performance. The fact that the choice of data points and the subset size is determined by our method ensures that each subset includes the data points that maximise the evaluation performance at each size. Our dataset pruning method has the advantage of determining the subset size along with the scores for the data points while previous work requires more fine-tuning runs to search for the optimal subset size after the score computation phase. We use the model’s output during the fine-tuning runs of the score computation phase to determine pruned subsets of the dataset. After creation, the pruned subsets can be used for faster fine-tuning runs.

3.3 EMPIRICAL ANALYSIS OF FINE-TUNING ON TRAINING SUBSETS

In this section, we study the effect of fine-tuning pre-trained models on subsets created using Equation 2. We first compute the $\mathcal{H}$ -scores of the training set of the fine-tuning task by fine-tuning it with 6 different seeds. We then create all the possible subsets, $\mathcal{D}_M$, and study the effects of using data points with certain $\mathcal{H}$ -scores on the evaluation performance of the model. Here we use the MNLI fine-tuning task with the OPT_{350M} model. Further examples of other models and tasks are available in Appendix A.2.

Figure 3 summarizes the results of these fine-tuning runs. It plots the accuracy of each run and the size of its training subset shown as a percentage of the full dataset. Based on these results, we divide the subsets into four groups:

1. The first group, shown with blue dots, is the set of training subsets, $\mathcal{D}_{M \cup \{6\}}$, for all $M \subset \{0, 1, \dots, 5\}$. In addition to other data points, subsets in this group always contain all of the data points with $\mathcal{H}_i = 6$. Given the number of data points with $\mathcal{H}_i = 6$ (Figure 2a), the size of these subsets is close to the full dataset. They also show consistently small accuracy drops compared to the full fine-tuning.
2. Distinguished with red dots, this group contains the set of training subsets, $\mathcal{D}_{M \cup \{0,5\}}$, for all $M \subseteq \{1, 2, \dots, 4\}$. Subsets in this group do not include data points with $\mathcal{H}_i = 6$, but contain the ones with $\mathcal{H}_i$ equal to 0 and 5. The drop in accuracy between the subsets in this group and previous group arises from excluding easy to learn data points, e.g. $\mathcal{H}_i = 6$, while keeping hard to learn ones.
3. These are the same subsets from the previous group where their data points with $\mathcal{H}_i = 5$ have also been removed. Shown with red circles, subsets in this group have a considerably lower accuracy compared to the ones in group 2. This accuracy drop indicates that data points with $\mathcal{H}_i = 5$ have an important role in stabilizing the training and maintaining the evaluation performance.
4. Subsets in this group, shown with green dots, do not contain any data point that has a $\mathcal{H}$ -score of either 0 or 6. These subsets generally benefit from the high accuracy that’s observed in group 1, while also being significantly smaller in terms of subset size. The few exceptional subsets of this group, with accuracies below 70%, are dominated by data points with a small $\mathcal{H}$ -score.

Throughout our experiments, subsets that included data points with larger $\mathcal{H}$ -scores better maintained the evaluation accuracy while data points with smaller $\mathcal{H}$ -scores negatively affected the evaluation accuracy of the model. To illustrate

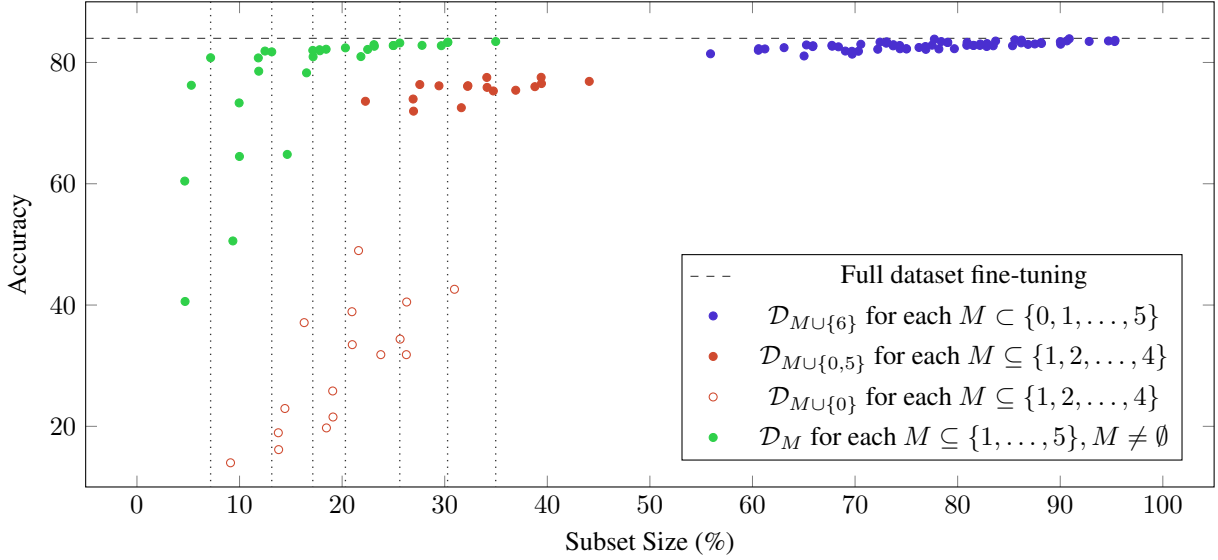


Figure 3: Subset size and evaluation accuracy of different subsets of the MNLI training set based on OPT_{350M}. Each dot represents a subset. Subsets are created using Equation 2 with all possible values of M . Our proposed subsets are noted with vertical dotted lines.

this point, consider the subset $\mathcal{D}_{\{1,2,3,4\}}$. The evaluation accuracy of the model fine-tuned on this subset is 81%. We then create $\mathcal{D}_{\{0,1,2,3,4\}}$ by adding all the data points with $\mathcal{H}_i = 0$ to this subset. Despite having more data points than the former, the evaluation accuracy of the latter subset drops to 43%. On the other hand, fine-tuning on $\mathcal{D}_{\{1,2,3,4,5\}}$ which contains data points with $\mathcal{H}_i = 5$ increases the evaluation accuracy to 83%. This observation is aligned with our decision to remove data points with smaller scores from our dataset pruning subsets.

Indicated with vertical dotted lines, our proposed dataset pruning subsets are found among the green dots of group 4. Since subsets of group 4 do not contain data points with a score of 6, our subsets tend to have a smaller size than subsets with similar accuracies. Also, our subsets have a better accuracy compared to other subsets of similar sizes. We observe a similar trend when plotting the evaluation performance for subsets of different fine-tuning tasks and models (Appendix A.2).

4 DATASET PRUNING EXPERIMENTS

4.1 EXPERIMENTAL SETUP

We apply our dataset pruning method on the training set of various fine-tuning tasks and use our proposed subsets to fine-tune pre-trained transformer-based language models. We use OPT_{350M} and RoBERTa_{LARGE} for our experiments. We add a task-specific head with randomly initialized parameters to the model with pre-trained parameters. In these experiments, we first create the pruned subsets for each pair of task and model and then use them for fine-tuning. The reported evaluation results are the average of 3 fine-tuning runs with different initialization seeds. The initialization seed controls the order of the data points and the initial parameters of the task-specific head. More implementation details and hyper-parameters are included in Appendix A.3.

To obtain our subsets, we fine-tune each of the models on each task using 6 different initialization seeds and use the outputs to compute the $\mathcal{H}$ -scores using Equation 1. The $\mathcal{H}$ -scores are then used to extract the training subsets as described in Section 3.2.

In addition to our subsets, we fine-tune the models on subsets that are obtained with a method similar to Swayamdipta et al. (2020). We first use the outputs of the same 6 fine-tuning runs to calculate their *variability* metric. Following the *ambiguous* setup of Swayamdipta et al. (2020) we create subsets by picking data points with the largest *variability* value. We create the same number of *ambiguous* subsets as ours. We set the subset sizes such that for each *ambiguous* subset there exists a subset of the same size created with our method. The *ambiguous* setup described here differs from the original paper because we use 6 runs, instead of 1 run, to calculate the *variability* score. Similarly, we create the

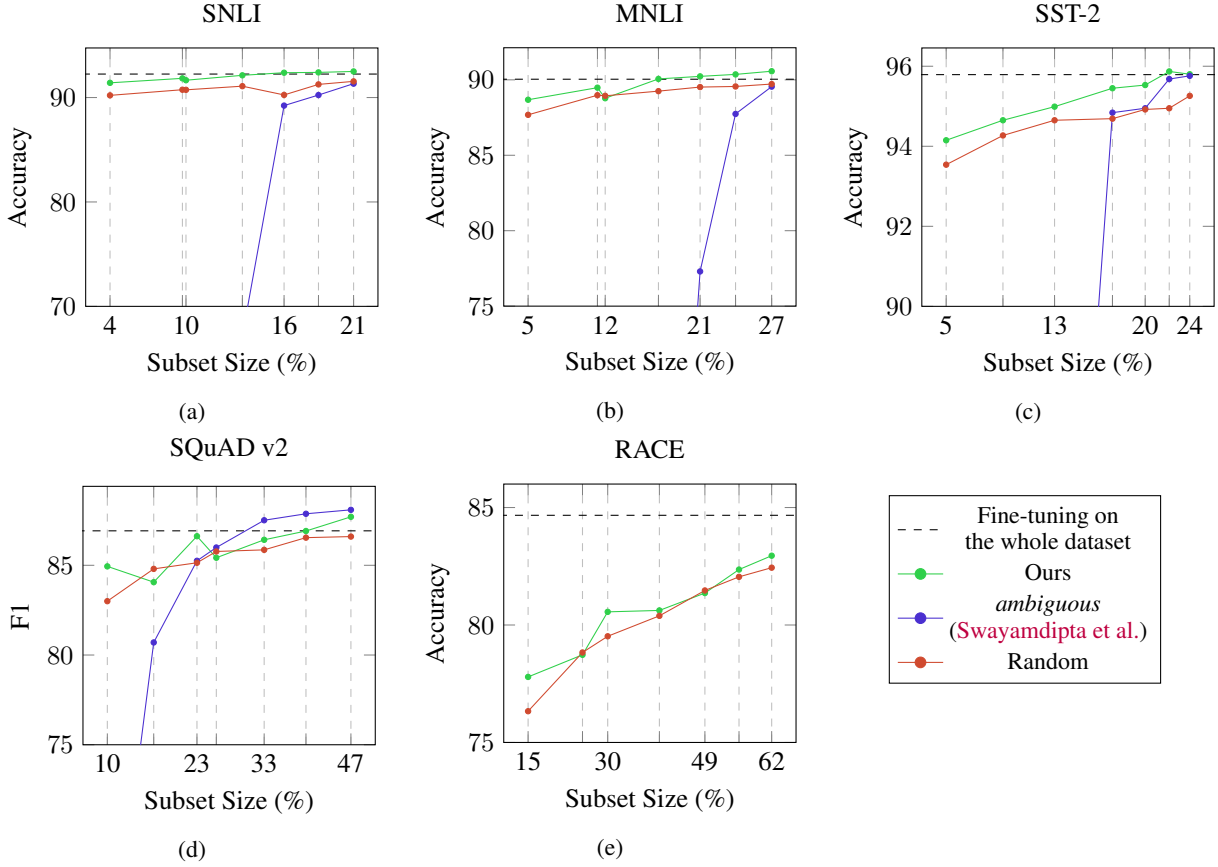


Figure 4: Evaluation accuracy of fine-tuning RoBERTa_{LARGE} on different subsets of the training dataset of multiple downstream tasks. All the plots share the same provided legend. In 4e the *ambiguous* setup falls outside the range of the vertical axis. Detailed accuracies and subset sizes are provided in Appendix A.4.

same number of randomly pruned subsets with each having the same number of data points as the *ambiguous* ones. To create a random subset of size x we shuffle the dataset and pick the first x data points.

We measure the evaluation metric performance of the models when fine-tuned on the pruned subsets along with the full training set. We run our fine-tuning experiments on five downstream tasks that target different natural language understanding abilities:

- **SNLI and MNLI:** The goal of these natural language inference tasks is to determine whether a given sentence entails, contradicts, or is neutral with respect to another sentence. Compared to SNLI (Bowman et al., 2015), MNLI is considered more challenging as it uses sentences from more diverse backgrounds (Williams et al., 2017).
- **SST-2:** The goal of this task is to classify whether each sentence has a negative or a positive sentiment (Socher et al., 2013).
- **SQuAD v2 and RACE:** Created for reading comprehension, the task in SQuAD v2 is to find the span of the answer of a question in the given passage. The second version of this dataset contains unanswerable questions which make it more challenging (Rajpurkar et al., 2018). RACE is another reading comprehension task which is made up of articles and multiple choice questions associated with them. The goal of the task is to find the correct answer choice for each question. The length of the articles and the fact that the questions are from real-world exams make RACE a more challenging reading comprehension task (Lai et al., 2017).

Except for RACE, the evaluation performance is measured using the publicly available validation sets of each task. For the RACE task, we use the publicly available test set to measure the evaluation performance.

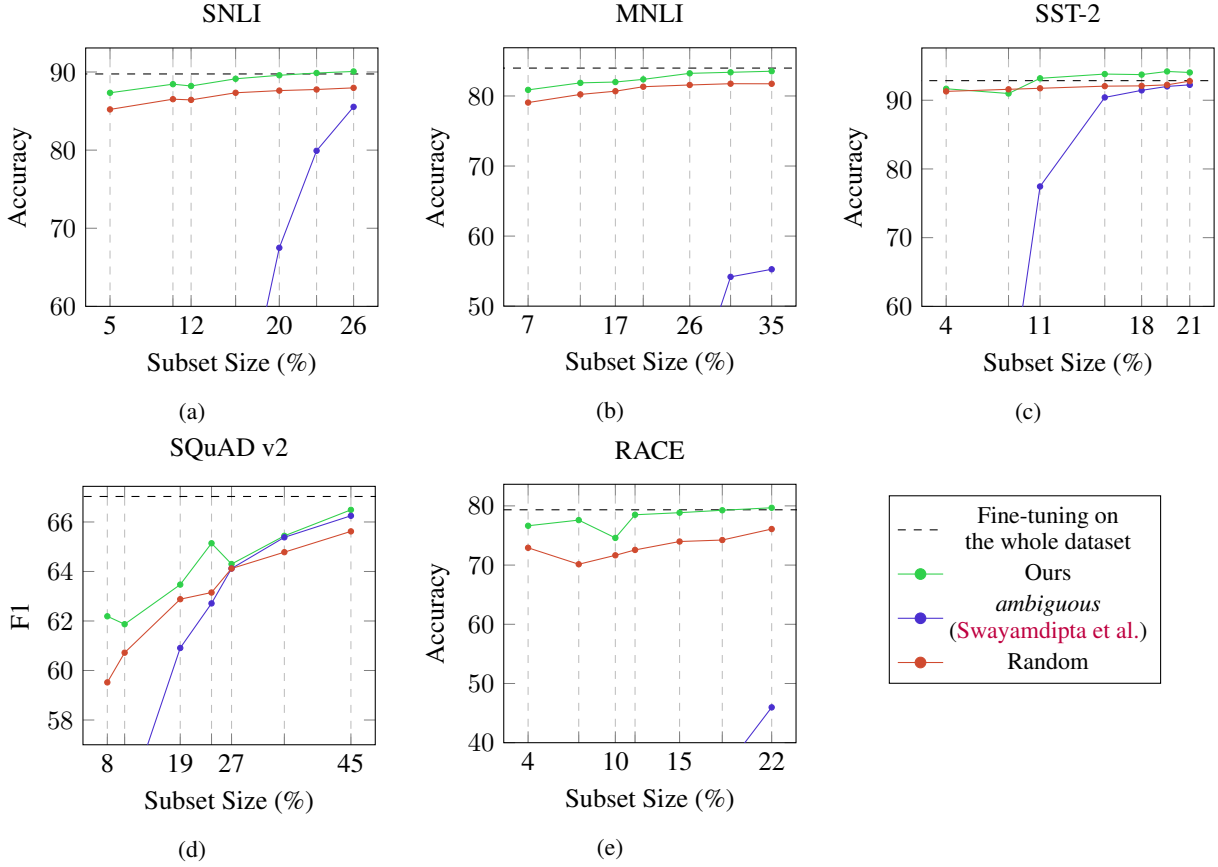


Figure 5: Evaluation accuracy of fine-tuning $\text{OPT}_{350\text{M}}$ on different subsets of the training dataset of multiple downstream tasks. All the plots share the same provided legend. Detailed accuracies and subset sizes are provided in Appendix A.4.

To compute the $\mathcal{H}$ -score for SQuAD v2 we consider the output as a correct classification if the span of the answer in the passage is exactly the same as the label.

4.2 RESULTS AND DISCUSSION

Figures 4 and 5 illustrate the results of our experiments on $\text{RoBERTa}_{\text{LARGE}}$ and $\text{OPT}_{350\text{M}}$, respectively. The evaluation metric performance is shown as F1 score for SQuAD v2 and as accuracy for other tasks. For each task, we plot the evaluation performance against the size of the training subset for different dataset pruning setups. We use vertical dashed lines to indicate each subset size and a horizontal dashed line to indicate the evaluation performance of full fine-tuning. The range of values in the vertical axis are limited for better visibility which resulted in some points falling outside the range, e.g. *ambiguous* subsets in Figure 4e. The values for accuracies and subset sizes are provided in Appendix A.4.

Our subsets contain significantly fewer data points than the training set of each task. Despite their small size, fine-tuning the models on these subsets results in a similar evaluation performance compared to full fine-tuning. The evaluation performance of our subsets shows that removing the hard-to-learn data points from the training set of the fine-tuning task does not negatively affect the model’s performance and in some cases increases the evaluation performance.

The number of data points in each subset, specified by our method, varies across different tasks and models. Our method tries to minimize the subset size for each case while achieving the highest evaluation performance. As different models have different behavior on the same task, using the same subset size for different cases results in suboptimal performance. For instance, despite having the same task, subset sizes of Figures 4e and 5e greatly vary, showing that

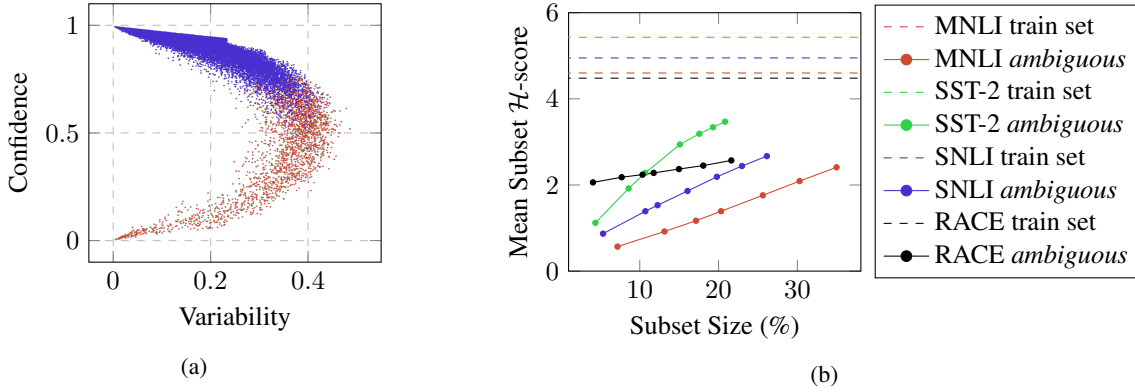


Figure 6: (a) Data map for the data points of the SST-2 train set, based on OPT_{350M}. Confidence and variability metrics are defined in Swayamdipta et al. (2020). Red dots represent data points with $\mathcal{H}_i \leq 1$ while other $\mathcal{H}$ -scores are shown as blue dots. (b) Mean $\mathcal{H}$ -score of training subsets along with the training set for MNLI, SST-2, and SNLI tasks, based on OPT_{350M}.

subset sizes from one model cannot be used for the other one. Our results also demonstrate the ability of our method in adapting the subset size for each model based on its properties.

In most cases, our subsets result in a higher evaluation performance than the ones created using the *ambiguous* method. This trend is reversed for the largest subsets in Figure 4d where our subsets have a slightly lower performance. We attribute this to the fact that our method negatively penalizes a slightly incorrect answer span which can result to unnecessary removal of data points. Overall, the low evaluation performance of the large *ambiguous* subsets in Figures 4e, 5b, and 5e along with their sudden performance drops for other tasks is indicative of the low reliability of the *ambiguous* method.

Figure 6a depicts the data map for the training set of the SST-2 task where data points with $\mathcal{H}_i \leq 1$ are distinguished with red dots. As the *ambiguous* subsets include data points with the largest *variability* value, they tend to be overpopulated with data points where $\mathcal{H}_i \leq 1$. Figure 6b depicts the mean $\mathcal{H}$ -score of the data points that are included in each *ambiguous* subset. A small mean $\mathcal{H}$ -score indicates that the subset is dominated by data points with a small $\mathcal{H}$ -score. Given the destabilizing effect of these points, discussed in Section 3.3, their high numbers in a subset leads to low evaluation accuracy. As the mean $\mathcal{H}$ -score increases, the evaluation accuracy of the *ambiguous* subsets in Figure 5 increases as well. For instance, mean $\mathcal{H}$ -score for SST-2 is increased with a higher rate than the MNLI. This rate difference is also reflected in their respective evaluation accuracies where the evaluation accuracy of smaller SST-2 subsets are closer to their baseline than MNLI subsets. Data maps for other tasks and models, and the mean $\mathcal{H}$ -score plot based on RoBERTa_{LARGE} are provided in Appendix A.5.

Across tasks and models, randomly created subsets closely follow our subsets in terms of evaluation performance. Considering that we utilize a uniform random function, the data points in these subsets have the same $\mathcal{H}$ -score distribution, and thus the same mean $\mathcal{H}$ -score, as the training set which helps them avoid bad evaluation performances. Despite this, random subsets are outperformed by our subsets in most cases which highlights the importance of excluding hard to learn data points from training. For instance, fine-tuning OPT_{350M} on the random subsets of the SNLI and RACE tasks results in an average drop in evaluation performance of 2% and 5%, respectively.

4.3 COMPARISON WITH FORGETTING SCORE

To compute the $\mathcal{H}$ -score, Equation 1 multiplies the reward of each data point across epochs and then adds the resulting value for multiple fine-tuning runs. As a result of the multiplication, only runs in which the data point is correctly classified in all epochs receive a reward of 1. More specifically, if a data point is learned in the middle of the run, i.e. the second epoch, and is never forgotten, it doesn't receive a reward of 1. This is in contrast with the *forgetting score* which rewards such data points and only assigns 0 rewards to forgetting events (Toneva et al., 2019). We create $\mathcal{F}$ -score to study the effect of the mentioned difference between $\mathcal{H}$ -score and forgetting score. $\mathcal{F}$ -score assigns a reward of 1 if a data point is learned either in the beginning or in the middle of a run, and not forgotten until the end.

Following Section 3.2, we use $\mathcal{F}$ -score to create subsets of the fine-tuning dataset and use them to fine-tune OPT_{350M} and RoBERTa_{LARGE}. Figure 17 of Appendix A.6 provides a comparison between the distribution of scores for both

methods. It shows a distribution shift in $\mathcal{F}$ -score subsets which results in more data points falling in the maximum score, leading to the pruning of more data points. As shown in Tables 6 and 7, the removed data points lead to an overall smaller size for subsets based on the $\mathcal{F}$ -score. However, fine-tuning the models on these subsets leads to significant drops in the evaluation performance compared to $\mathcal{H}$ -score subsets. As a result, using $\mathcal{F}$ -score, and thus following Toneva et al. (2019), results in subsets with a lower evaluation performance compared to $\mathcal{H}$ -score.

4.4 SENSITIVITY ANALYSIS ON E AND S HYPERPARAMETERS

The number of fine-tuning runs that are used to compute the $\mathcal{H}$ -score is shown as S in Equation 1. We study the sensitivity of our dataset pruning method to the value of S . We create winning ticket subsets with $\mathcal{H}$ -scores for all possible values of $S \in \{2, 3, 4, 5, 6\}$. As shown in Table 8 of Appendix A.7, both the winning ticket subset size and the evaluation performance of the model drops when S is decreased. Compared to subsets of similar size with $S = 6$, winning ticket subsets with $S < 6$ sometimes have a slightly higher evaluation performance, e.g. RACE on RoBERTa_{LARGE} (Tables 2 to 4). However, as $\mathcal{H}_i \in \{0, 1, \dots, S\}$, a larger S leads to a higher granularity of the $\mathcal{H}$ -score and more options for subsets. Moreover, extreme cases like $S = 2$ lack sufficient information about the effect of the initialization seed due to the limited number of outputs used for score computation. A similar trend is observed in the case of E , i.e. the number of epochs used to compute the $\mathcal{H}$ -score. Tables 9 and 10 of Appendix A.7 show that decreasing the hyperparameter E from 3 to 1 has an overall negative effect on the evaluation performance of the model.

Our sensitivity analysis experiments provide information for the trade-off between the overhead of computing the $\mathcal{H}$ -score and the evaluation performance of the created subsets. Although reducing the number of epochs and fine-tuning runs mitigates the overhead of dataset pruning, it can also result in subsets with a worse evaluation performance.

5 CONCLUSION AND FUTURE WORK

We presented a novel method to prune the training set of fine-tuning tasks of the transformer-based language models. We propose a dataset difficulty score, $\mathcal{H}$ -score, which is based on the predictions of the model for each training data point. Using the $\mathcal{H}$ -score, we create subsets of the training set that are used for fine-tuning the model. Unlike other dataset pruning methods, our method creates subsets that are optimized for each model. Such design enables automatic dataset pruning and avoids the costly hyper-parameter search. Data points in each subset are selected to minimize subset size and maximise the evaluation performance of the model that is fine-tuned on them. Our experiments show that our largest subset, the winning ticket subset, is on average 33% of the training set but results in a 0.1% increase in the evaluation performance of the fine-tuned model. Given their evaluation performance, the pruned subsets can replace the full fine-tuning dataset and save time and resources in applications where multiple fine-tuning runs are needed, e.g. neural architecture search.

For future work, we will apply our method to a wider variety of natural language understanding tasks. Specifically, we will focus on question answering tasks similar to SQuAD. In addition to that, the effect of fine-tuning on the pruned subset on the evaluation performance of the model on adversarial evaluation sets is a topic of interest which is left for future work.

6 LIMITATIONS

Through removing parts of the training set, our method essentially limits the information that is fed to a pre-trained model during fine-tuning. Our experiments have shown that the model is able to maintain its performance on the validation or the test set associated with each task. However, these datasets may not be capable to fully capture the model’s ability in learning the fine-tuning task. As a result, the performance of our fine-tuned models on adversarial tasks can be lower than full fine-tuning. This topic is left for future work.

Although our method is effective in achieving near baseline evaluation performance, the size of the winning ticket subset varies across different models. As a result, the size of the pruned subsets, and thus the speedup associated with it, is dependant on the model and the task. We leave the topic of studying the effectiveness of our proposed method on more models and tasks to future work.

ACKNOWLEDGMENTS

We would like to thank our colleagues Amir Ardakani and Hugo Tessier, and the anonymous reviewers for their helpful comments and discussions. This work was supported by Huawei Technologies Canada.

REFERENCES

- Robert Baldock, Hartmut Maennel, and Behnam Neyshabur. Deep learning through the lens of example difficulty. *Advances in Neural Information Processing Systems*, 34:10876–10889, 2021.
- Samuel R Bowman, Gabor Angeli, Christopher Potts, and Christopher D Manning. A large annotated corpus for learning natural language inference. *arXiv preprint arXiv:1508.05326*, 2015.
- Daoyuan Chen, Yaliang Li, Minghui Qiu, Zhen Wang, Bofang Li, Bolin Ding, Hongbo Deng, Jun Huang, Wei Lin, and Jingren Zhou. Adabert: Task-adaptive bert compression with differentiable neural architecture search. *arXiv preprint arXiv:2001.04246*, 2020.
- Nadir Durrani, Hassan Sajjad, Fahim Dalvi, and Firoj Alam. On the transformation of latent space in fine-tuned NLP models. In Yoav Goldberg, Zornitsa Kozareva, and Yue Zhang (eds.), *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pp. 1495–1516, Abu Dhabi, United Arab Emirates, December 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.emnlp-main.97. URL <https://aclanthology.org/2022.emnlp-main.97>.
- Kawin Ethayarajh, Yejin Choi, and Swabha Swayamdipta. Understanding dataset difficulty with v-usable information. In *International Conference on Machine Learning*, pp. 5988–6008. PMLR, 2022.
- Mohsen Fayyaz, Ehsan Aghazadeh, Ali Modarressi, Mohammad Taher Pilehvar, Yadollah Yaghoobzadeh, and Samira Ebrahimi Kahou. Bert on a data diet: Finding important examples by gradient-based pruning. *arXiv preprint arXiv:2211.05610*, 2022.
- Angelos Katharopoulos and François Fleuret. Not all samples are created equal: Deep learning with importance sampling. In *International conference on machine learning*, pp. 2525–2534. PMLR, 2018.
- Guokun Lai, Qizhe Xie, Hanxiao Liu, Yiming Yang, and Eduard Hovy. Race: Large-scale reading comprehension dataset from examinations. *arXiv preprint arXiv:1704.04683*, 2017.
- Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.
- Mansheej Paul, Surya Ganguli, and Gintare Karolina Dziugaite. Deep learning on a data diet: Finding important examples early in training. *Advances in Neural Information Processing Systems*, 34:20596–20607, 2021.
- Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *The Journal of Machine Learning Research*, 21(1):5485–5551, 2020.
- Pranav Rajpurkar, Robin Jia, and Percy Liang. Know what you don’t know: Unanswerable questions for squad. *arXiv preprint arXiv:1806.03822*, 2018.
- Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D Manning, Andrew Y Ng, and Christopher Potts. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of the 2013 conference on empirical methods in natural language processing*, pp. 1631–1642, 2013.
- Ben Sorscher, Robert Geirhos, Shashank Shekhar, Surya Ganguli, and Ari Morcos. Beyond neural scaling laws: beating power law scaling via data pruning. *Advances in Neural Information Processing Systems*, 35:19523–19536, 2022.
- Swabha Swayamdipta, Roy Schwartz, Nicholas Lourie, Yizhong Wang, Hannaneh Hajishirzi, Noah A. Smith, and Yejin Choi. Dataset cartography: Mapping and diagnosing datasets with training dynamics. In Bonnie Webber, Trevor Cohn, Yulan He, and Yang Liu (eds.), *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 9275–9293, Online, November 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.emnlp-main.746. URL <https://aclanthology.org/2020.emnlp-main.746>.
- Mariya Toneva, Alessandro Sordoni, Remi Tachet des Combes, Adam Trischler, Yoshua Bengio, and Geoffrey J. Gordon. An empirical study of example forgetting during deep neural network learning. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=BJlxm30cKm>.

Kailas Vodrahalli, Ke Li, and Jitendra Malik. Are all training examples created equal? an empirical study. *arXiv preprint arXiv:1811.12569*, 2018.

Adina Williams, Nikita Nangia, and Samuel R Bowman. A broad-coverage challenge corpus for sentence understanding through inference. *arXiv preprint arXiv:1704.05426*, 2017.

Jin Xu, Xu Tan, Renqian Luo, Kaitao Song, Jian Li, Tao Qin, and Tie-Yan Liu. Nas-bert: Task-agnostic and adaptive-size bert compression with neural architecture search. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*, pp. 1933–1943, 2021.

Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, et al. Opt: Open pre-trained transformer language models. *arXiv preprint arXiv:2205.01068*, 2022.

A APPENDIX

A.1 $\mathcal{H}$ -SCORE DISTRIBUTION FOR OTHER TASKS

The $\mathcal{H}$ -score distributions of the SNLI, SST-2, and RACE tasks are provided in Figure 7.

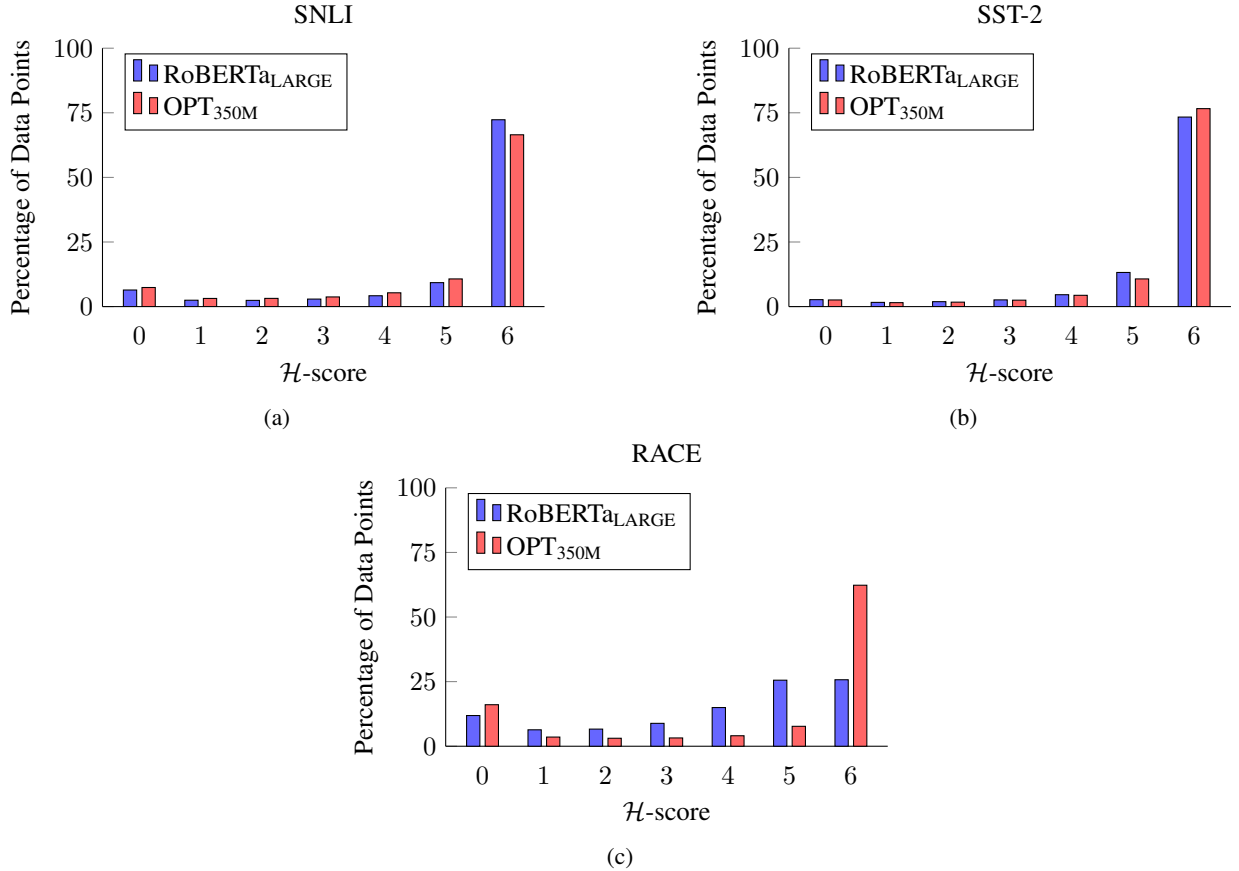


Figure 7: Distribution of the $\mathcal{H}$ -score for the training set of the SNLI, SST-2, and RACE tasks, based on RoBERTa_{LARGE} and OPT_{350M}.

A.2 SUBSETS FOR OTHER TASKS AND MODELS

Figures 8, 9, 10, 11, and 12 plot the evaluation accuracy against the subset size for all possible subsets created using Equation 2 for various tasks and models.

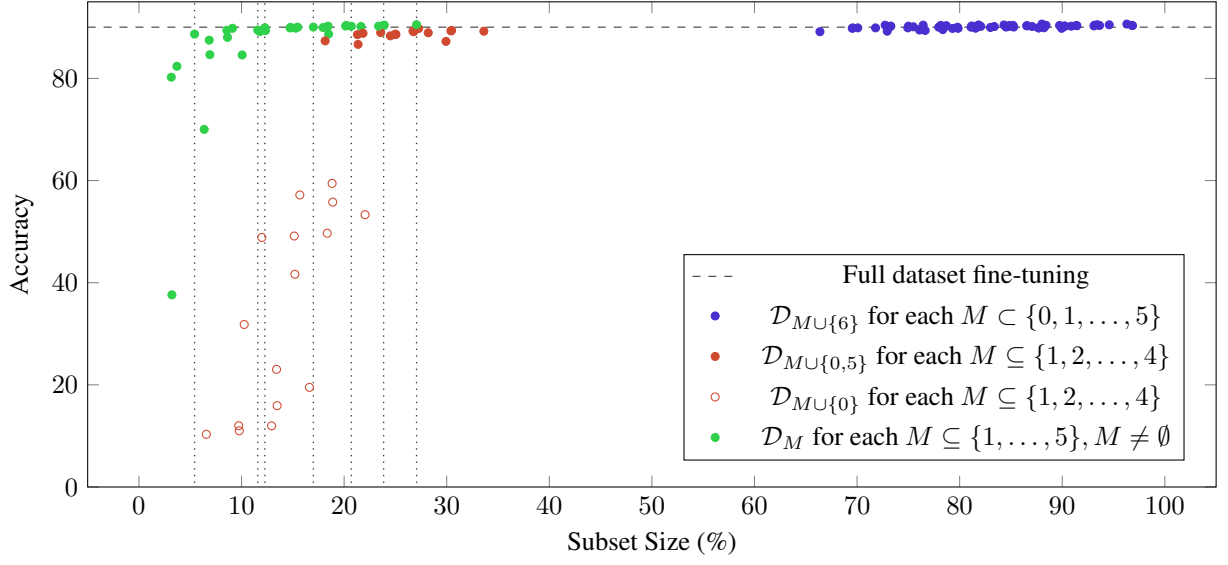


Figure 8: Subset size and evaluation accuracy of different subsets of the MNLI training set based on RoBERTa_{LARGE}. Our proposed subsets are noted with vertical dotted lines.

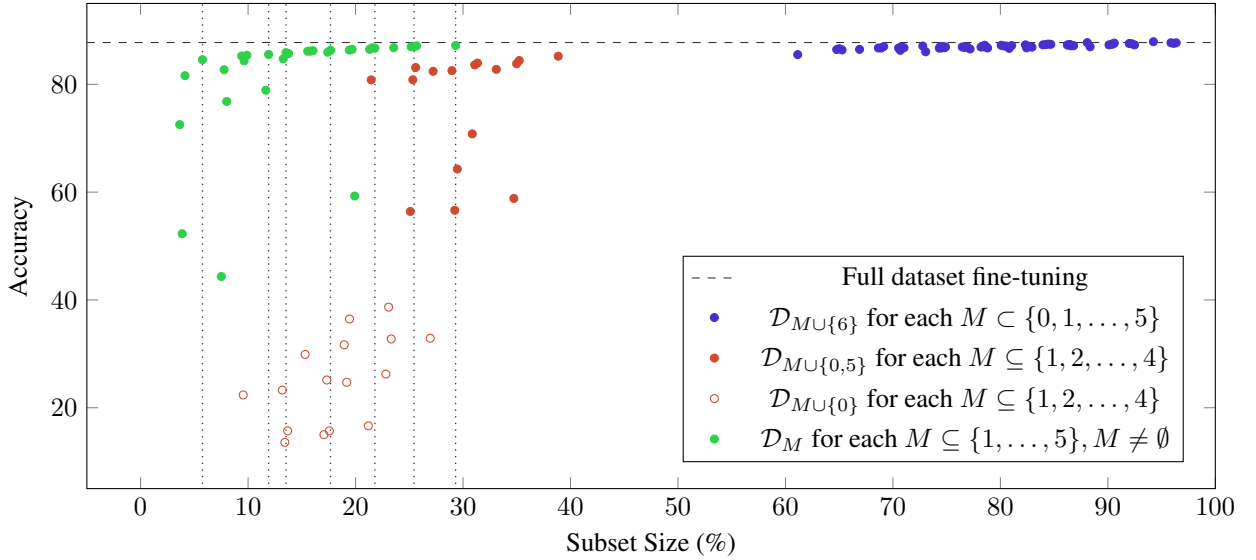


Figure 9: Subset size and evaluation accuracy of different subsets of the MNLI training set based on RoBERTa_{BASE}. Our proposed subsets are noted with vertical dotted lines.

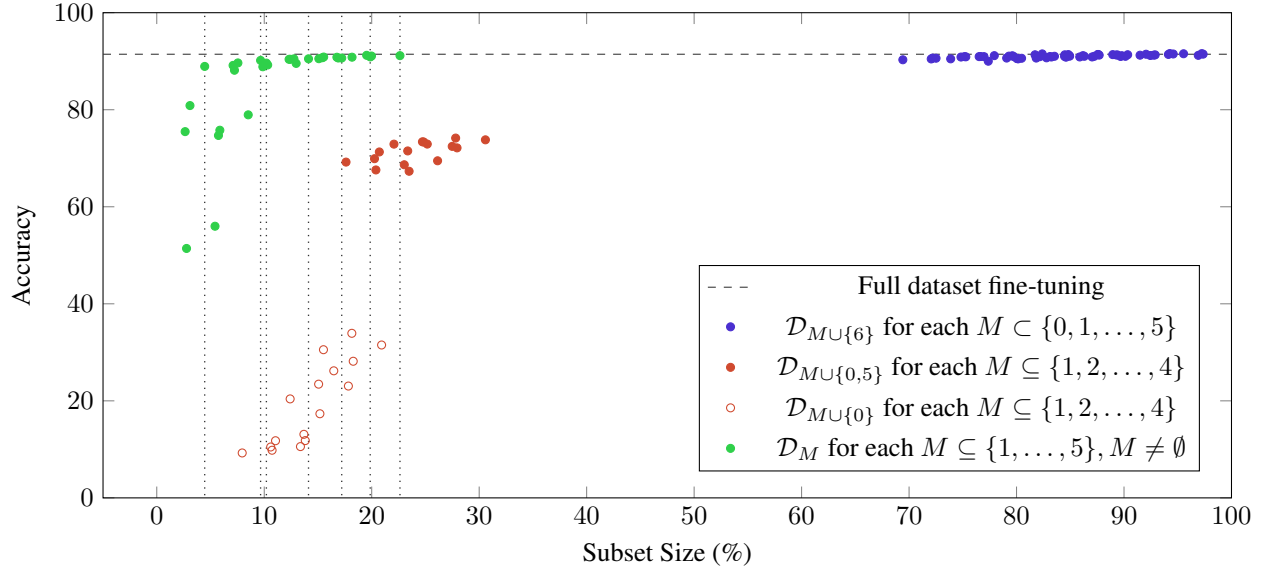


Figure 10: Subset size and evaluation accuracy of different subsets of the SNLI training set based on RoBERTa_{BASE}. Our proposed subsets are noted with vertical dotted lines.

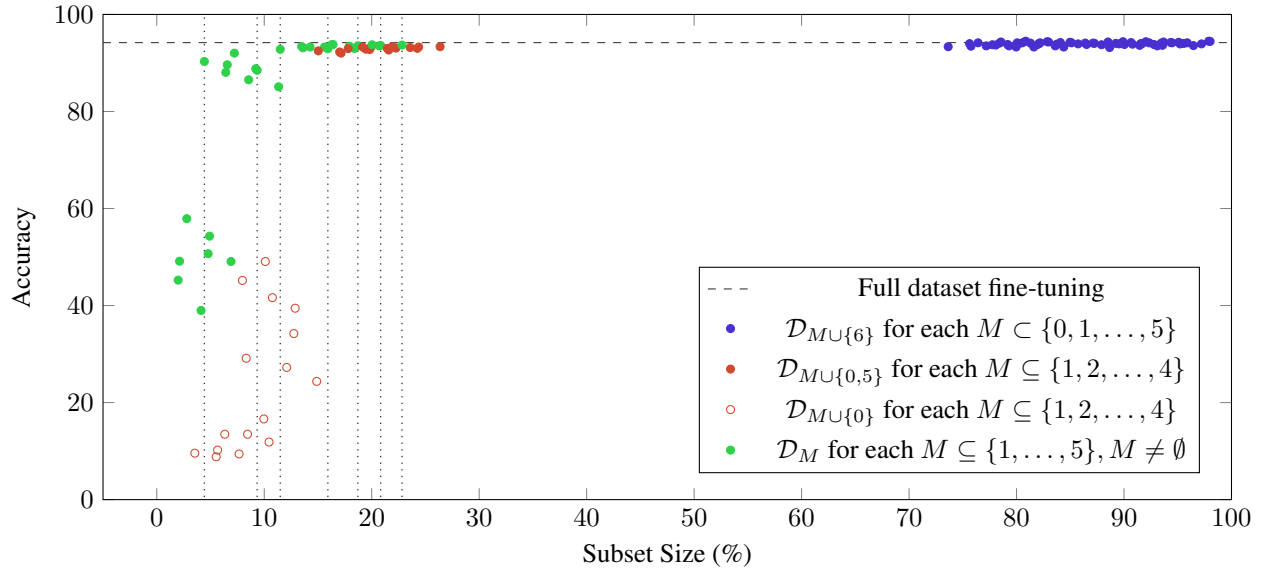


Figure 11: Subset size and evaluation accuracy of different subsets of the SST-2 training set based on RoBERTa_{BASE}. Our proposed subsets are noted with vertical dotted lines.

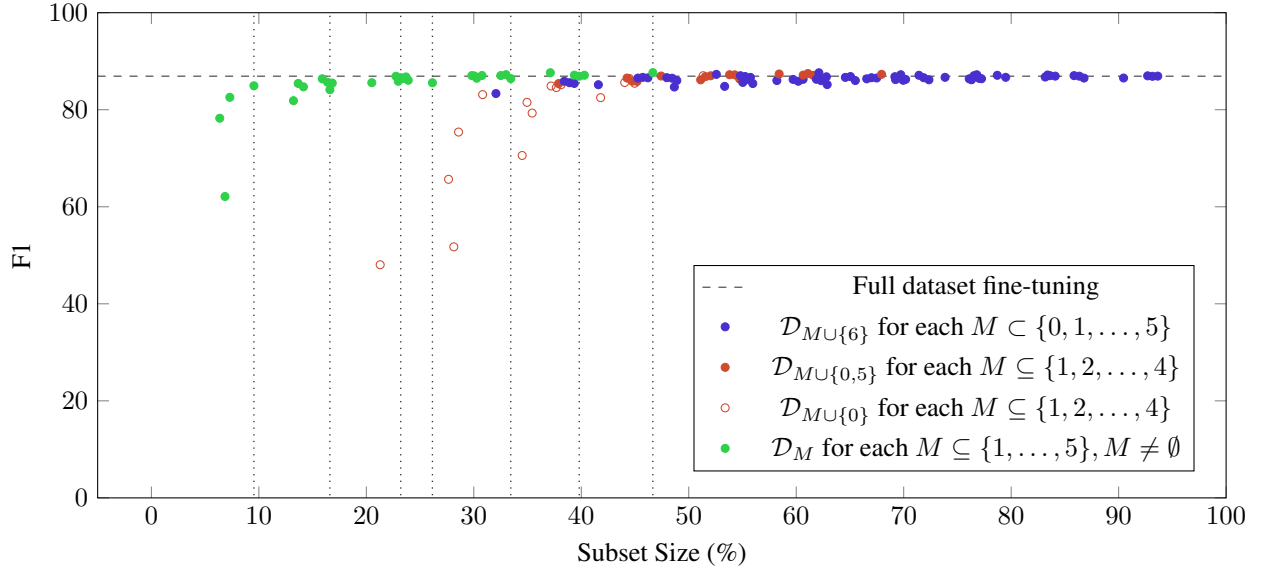


Figure 12: Subset size and evaluation accuracy of different subsets of the SQuAD v2 training set based on RoBERTa_{LARGE}. Our proposed subsets are noted with vertical dotted lines.

A.3 DETAILS OF EXPERIMENTAL SETUP

Our experiments are created on top of the existing fine-tuning codes in version 4.31.0 of the transformers library². Specifically, for the MNLI, SNLI, and SST-2 tasks we use `run_glue.py`, for the RACE task we use `run_swag.py`, and for SQuAD v2 we use `run_qa.py`. All our experiments are done on an NVIDIA RTX 4090 GPU with version 12.1 of the CUDA framework and version 2.1.0 of PyTorch.

We use the `RobertaForMultipleChoice` class for fine-tuning RoBERTa_{LARGE} on RACE. For OPT_{350M}, we use the `OPTForSequenceClassification` class and treat RACE as a sequence classification task. To do this, we concatenate each choice to its corresponding article and question with the label set to true for the correct choice.

Table 1 contains the list of hyper-parameters used in the fine-tuning experiments.

		Learning Rate	Batch Size	Gradient Accumulation Steps	Max Sequence Length	Fine-tuning Epochs
RoBERTa _{Large}	MNLI	10^{-5}	32	1	128	3
	SNLI	10^{-5}	32	1	128	3
	SST-2	10^{-5}	32	1	128	3
	RACE	10^{-5}	2	8	512	3
	SQuAD v2	3×10^{-5}	12	1	384	2
OPT _{350M}	MNLI	10^{-5}	32	1	128	3
	SNLI	10^{-5}	32	1	128	3
	SST-2	10^{-5}	32	1	128	3
	RACE	10^{-5}	8	2	512	3
	SQuAD v2	3×10^{-5}	12	1	384	2

Table 1: Values for hyper-parameters used in the fine-tuning experiments. Any hyper-parameter which is not mentioned is set to the default value of the framework.

²<https://github.com/huggingface/transformers/>

A.4 EXPERIMENTAL RESULTS

Tables 2, 3, and 4 demonstrate the evaluation performance of our fine-tuning experiments described in Section 4. The subsets are distinguished by their size. Table 5 provides the size of each subset discussed in Section 3.2.

MNLI Matched (Accuracy)									
		Subset Size							
		5.42%	11.58%	12.27%	17.0%	20.7%	23.85%	27.06%	100%
RoBERTa Large	Ours	88.68 _{0.50}	89.48 _{0.09}	89.91 _{0.24}	90.06 _{0.13}	90.23 _{0.11}	90.36 _{0.09}	90.57 _{0.12}	90.04 _{0.08}
	<i>ambiguous</i>	26.03 _{6.14}	31.23 _{0.85}	31.59 _{1.43}	37.56 _{9.94}	77.30 _{1.86}	87.74 _{0.39}	89.55 _{0.05}	
	Random	87.68 _{0.36}	88.98 _{0.12}	88.94 _{0.28}	89.25 _{0.22}	89.52 _{0.08}	89.56 _{0.31}	89.72 _{0.34}	
		Subset Size							
		7.18%	13.14%	17.14%	20.32%	25.62%	30.28%	34.97%	100%
OPT 350M	Ours	80.87 _{0.39}	81.87 _{0.14}	81.99 _{0.44}	82.37 _{0.19}	83.23 _{0.14}	83.38 _{0.11}	83.53 _{0.37}	83.98 _{0.18}
	<i>ambiguous</i>	19.12 _{0.56}	22.17 _{0.19}	23.90 _{0.38}	27.74 _{0.24}	36.65 _{4.28}	54.18 _{0.93}	55.26 _{20.34}	
	Random	79.06 _{0.48}	80.22 _{0.37}	80.68 _{0.31}	81.32 _{0.36}	81.57 _{0.14}	81.75 _{0.19}	81.74 _{0.23}	
MNLI Mismatched (Accuracy)									
		Subset Size							
		5.42%	11.58%	12.27%	17.0%	20.7%	23.85%	27.06%	100%
RoBERTa Large	Ours	88.79 _{0.35}	89.26 _{0.25}	89.61 _{0.18}	89.81 _{0.14}	90.05 _{0.04}	90.12 _{0.31}	90.22 _{0.05}	89.99 _{0.06}
	<i>ambiguous</i>	26.28 _{6.14}	30.22 _{1.47}	30.22 _{1.87}	37.76 _{10.29}	76.53 _{2.12}	87.74 _{0.39}	89.27 _{0.17}	
	Random	87.79 _{0.14}	88.61 _{0.14}	88.74 _{0.15}	89.16 _{0.37}	89.12 _{0.30}	89.20 _{0.31}	89.27 _{0.34}	
		Subset Size							
		7.18%	13.14%	17.14%	20.32%	25.62%	30.28%	34.97%	100%
OPT 350M	Ours	81.83 _{0.35}	82.56 _{0.25}	82.85 _{0.18}	83.55 _{0.14}	83.70 _{0.04}	84.15 _{0.31}	84.29 _{0.05}	83.98 _{0.18}
	<i>ambiguous</i>	17.68 _{1.11}	20.47 _{0.34}	22.28 _{0.30}	25.63 _{0.74}	35.66 _{3.34}	52.17 _{0.82}	54.31 _{19.52}	
	Random	79.81 _{0.10}	81.11 _{0.16}	81.52 _{0.15}	81.68 _{0.58}	82.08 _{0.36}	82.37 _{0.44}	82.53 _{0.13}	
SNLI (Accuracy)									
		Subset Size							
		4.2%	9.27%	9.52%	13.47%	16.39%	18.8%	21.26%	100%
RoBERTa Large	Ours	91.41 _{0.31}	91.84 _{0.15}	91.66 _{0.16}	92.13 _{0.22}	92.38 _{0.03}	92.41 _{0.03}	92.50 _{0.14}	92.25 _{0.12}
	<i>ambiguous</i>	30.89 _{3.57}	58.45 _{4.81}	61.80 _{5.09}	67.18 _{29.80}	89.23 _{0.25}	90.24 _{1.05}	91.33 _{0.13}	
	Random	90.21 _{0.32}	90.75 _{0.28}	90.74 _{0.06}	91.09 _{0.35}	90.25 _{2.17}	91.26 _{0.22}	91.55 _{0.05}	
		Subset Size							
		5.34%	10.71%	12.26%	16.06%	19.79%	22.97%	26.13%	100%
OPT 350M	Ours	87.34 _{0.12}	88.44 _{0.03}	88.21 _{0.32}	89.13 _{0.13}	89.58 _{0.11}	89.87 _{0.14}	90.07 _{0.29}	89.75 _{0.21}
	<i>ambiguous</i>	17.16 _{0.16}	21.57 _{0.48}	24.22 _{1.07}	41.35 _{3.83}	67.50 _{1.84}	79.92 _{1.01}	85.54 _{0.85}	
	Random	85.21 _{0.29}	86.52 _{0.09}	86.43 _{0.06}	87.34 _{0.19}	87.62 _{0.10}	87.76 _{0.09}	87.97 _{0.21}	

Table 2: Evaluation accuracy of fine-tuning RoBERTa_{LARGE} and OPT_{350M} on various training subsets. For each subset size, the best result is indicated with a bold font. Results that outperform the baseline are indicated with underline. Each reported accuracy is the average of 3 runs with different initialization seeds. The standard deviation of the runs is also reported as the subscript. Last column reports the accuracy of fine-tuning on the entire dataset (baseline).

SST-2 (Accuracy)									
		Subset Size							
		4.6%	9.11%	13.21%	17.81%	20.41%	22.32%	23.95%	100%
RoBERTa Large	Ours	94.15 _{0.35}	94.65 _{0.46}	94.99 _{0.17}	95.45 _{0.24}	95.53 _{0.42}	95.87 _{0.23}	95.80 _{0.37}	95.79 _{0.58}
	<i>ambiguous</i>	50.69 _{0.40}	50.54 _{1.48}	74.62 _{20.33}	94.84 _{0.50}	94.95 _{1.05}	95.68 _{0.40}	95.76 _{0.42}	
	Random	93.54 _{0.52}	94.27 _{0.46}	94.65 _{0.76}	94.69 _{0.37}	94.92 _{0.58}	94.95 _{0.35}	95.26 _{0.63}	
		Subset Size							
		4.37%	8.58%	10.71%	15.08%	17.57%	19.29%	20.82%	100%
OPT 350M	Ours	91.67 _{0.33}	90.98 _{0.35}	93.19 _{0.18}	93.81 _{0.12}	93.73 _{0.07}	94.19 _{0.29}	94.04 _{0.12}	92.85 _{0.52}
	<i>ambiguous</i>	16.63 _{4.54}	45.68 _{18.05}	77.45 _{3.39}	90.41 _{0.37}	91.44 _{1.07}	92.01 _{0.35}	92.24 _{0.24}	
	Random	91.29 _{0.50}	91.59 _{0.87}	91.74 _{0.59}	92.05 _{0.48}	92.09 _{0.20}	92.24 _{0.95}	92.78 _{0.52}	
RACE (Accuracy)									
		Subset Size							
		14.97%	25.56%	30.47%	40.53%	49.39%	56.03%	62.39%	100%
RoBERTa Large	Ours	77.79 _{0.39}	78.73 _{0.25}	80.56 _{0.73}	80.62 _{0.12}	81.36 _{0.43}	82.36 _{0.54}	82.95 _{0.13}	84.67 _{0.17}
	<i>ambiguous</i>	24.02 _{1.25}	23.56 _{0.95}	23.68 _{1.43}	43.58 _{33.44}	62.47 _{34.48}	24.36 _{0.96}	43.21 _{34.81}	
	Random	76.33 _{0.84}	78.83 _{0.44}	79.52 _{0.64}	80.39 _{0.44}	81.47 _{0.19}	82.05 _{0.44}	82.44 _{0.34}	
		Subset Size							
		4.06%	7.71%	10.35%	11.77%	14.98%	18.06%	21.62%	100%
OPT 350M	Ours	76.64 _{0.42}	77.60 _{0.25}	74.59 _{0.20}	78.50 _{0.10}	78.85 _{0.06}	79.26 _{0.25}	79.68 _{0.09}	79.34 _{0.20}
	<i>ambiguous</i>	33.30 _{4.03}	34.60 _{4.05}	34.34 _{5.99}	36.07 _{8.37}	32.99 _{0.30}	35.00 _{2.55}	45.97 _{4.04}	
	Random	72.92 _{2.54}	70.14 _{1.45}	71.65 _{0.86}	72.56 _{2.78}	73.98 _{2.01}	74.23 _{0.87}	76.09 _{0.46}	

Table 3: Evaluation accuracy of fine-tuning RoBERTa_{LARGE} and OPT_{350M} on various training subsets. For each subset size, the best result is indicated with a bold font. Results that outperform the baseline are indicated with underline. Each reported accuracy is the average of 3 runs with different initialization seeds. The standard deviation of the runs is also reported as the subscript. Last column reports the accuracy of fine-tuning on the entire dataset (baseline).

		SQuAD v2 (Exact Match F1 Score)							
		Subset Size							
		9.54%	16.61%	23.2%	26.15%	33.44%	39.81%	46.66%	100%
RoBERTa Large	Ours	82.13 _{0.20}	81.40 _{0.20}	84.03 _{0.34}	82.84 _{0.41}	83.80 _{0.22}	<u>84.39</u> _{0.25}	<u>85.09</u> _{0.14}	83.94 _{0.62} 86.92 _{0.66}
		84.94 _{0.07}	84.06 _{0.28}	86.62 _{0.26}	85.42 _{0.45}	86.42 _{0.30}	<u>86.91</u> _{0.28}	<u>87.70</u> _{0.17}	
	<i>ambiguous</i>	60.11 _{1.30}	78.87 _{0.67}	82.90 _{0.40}	83.57 _{0.38}	84.93 _{0.07}	85.19 _{0.38}	85.32 _{0.15}	
		60.51 _{1.39}	80.70 _{0.65}	85.25 _{0.40}	85.99 _{0.37}	87.51 _{0.06}	87.87 _{0.36}	88.09 _{0.10}	
	Random	79.65 _{0.92}	81.60 _{0.44}	81.96 _{0.60}	82.70 _{0.90}	82.71 _{0.07}	83.44 _{0.28}	83.52 _{0.10}	
		83.00 _{0.83}	84.80 _{0.61}	85.14 _{0.59}	85.77 _{0.90}	85.86 _{0.10}	86.54 _{0.27}	86.60 _{0.11}	
		Subset Size							
		8.38%	10.99%	19.38%	24.10%	27.12%	35.09%	45.13%	100%
OPT 350M	Ours	59.75 _{0.34}	59.54 _{0.17}	61.00 _{0.14}	62.50 _{0.45}	61.77 _{0.21}	62.86 _{0.12}	63.72 _{0.38}	63.26 _{0.31} 67.03 _{0.23}
		62.19 _{0.19}	61.87 _{0.20}	63.47 _{0.10}	65.14 _{0.34}	64.30 _{0.31}	65.44 _{0.13}	66.49 _{0.22}	
	<i>ambiguous</i>	48.97 _{0.35}	51.59 _{1.06}	57.90 _{0.54}	59.66 _{0.68}	61.02 _{0.70}	62.30 _{0.36}	62.98 _{0.41}	
		51.30 _{0.50}	54.25 _{1.00}	60.91 _{0.22}	62.71 _{0.47}	64.12 _{0.52}	65.38 _{0.28}	66.25 _{0.43}	
	Random	55.19 _{0.50}	56.37 _{0.74}	58.87 _{0.88}	59.17 _{0.38}	60.05 _{0.15}	60.76 _{0.08}	61.66 _{0.27}	
		59.52 _{0.28}	60.72 _{0.68}	62.88 _{0.73}	63.15 _{0.28}	64.12 _{0.18}	64.78 _{0.17}	65.62 _{0.13}	

Table 4: Evaluation accuracy of fine-tuning RoBERTa_{LARGE} and OPT_{350M} on training subsets of SQuAD v2 task. For each subset size, the best result is indicated with a bold font. Results that outperform the baseline are indicated with underline. Each reported metric is the average of 3 runs with different initialization seeds. The standard deviation of the runs is also reported as the subscript. Last column reports the accuracy of fine-tuning on the entire dataset (baseline).

		Subset						
		$\mathcal{D}_{\{2,3,4\}}$	$\mathcal{D}_{\{4\}}$	$\mathcal{D}_{\{5\}}$	$\mathcal{D}_{\{4,5\}}$	$\mathcal{D}_{\{3,4,5\}}$	$\mathcal{D}_{\{2,3,4,5\}}$	$\mathcal{D}_{\{1,2,3,4,5\}}$
RoBERTa Large	MNLI	12.27%	5.42%	11.58%	17.00%	20.70%	23.85%	27.06%
	SNLI	9.52%	4.2%	9.27%	13.47%	16.39%	18.8%	21.26%
	SST-2	9.11%	4.6%	13.21%	17.81%	20.41%	22.32%	23.95%
	RACE	30.47%	14.97%	25.26%	40.53%	49.39%	56.03%	62.39%
	SQuAD v2	23.2%	9.54%	16.61%	26.15%	33.44%	39.81%	46.66%
OPT 350M	MNLI	17.14%	7.18%	13.14%	20.32%	25.62%	30.28%	34.97%
	SNLI	12.26%	5.34%	10.71%	16.06%	19.79%	22.97%	26.13%
	SST-2	8.58%	4.37%	10.71%	15.08%	17.57%	19.29%	20.82%
	RACE	10.35%	4.06%	7.71%	11.77%	14.98%	18.06%	21.62%
	SQuAD v2	19.38%	8.38%	10.99%	24.1%	27.12%	35.09%	45.13%

Table 5: Relative size for each of our proposed subsets used for dataset pruning compared to the training set. $\mathcal{D}_{\{1,2,3,4,5\}}$ is also referred to as the winning ticket subset.

A.5 FURTHER COMPARISON WITH *ambiguous* SUBSETS

Similar to Figure 6b, Figure 13 depicts the mean $\mathcal{H}$ -score against the subset size for various tasks based on RoBERTa_{LARGE}. Figures 14, 15, and 16 depict the data maps for the training sets of all our tasks. We compute the variability and confidence metric on the outputs of our fine-tuning runs, following Swayamdipta et al. (2020). Variability is calculated as the standard deviation of the probability of the model for the golden label and confidence is its average.

In each plot, red dots represent data points with $\mathcal{H}_i \in \{0, 1\}$, green dots represent data points with $\mathcal{H}_i \in \{2, 3, 4\}$, and blue dots represent data points with $\mathcal{H}_i \in \{5, 6\}$. [Swayamdipta et al. \(2020\)](#) considers data points with a large variability metric as the most ambiguous. However, we believe data points with $\mathcal{H}_i \in \{2, 3, 4\}$ are the most ambiguous. Figures 14, 15, and 16 clearly show the difference between our two methods.

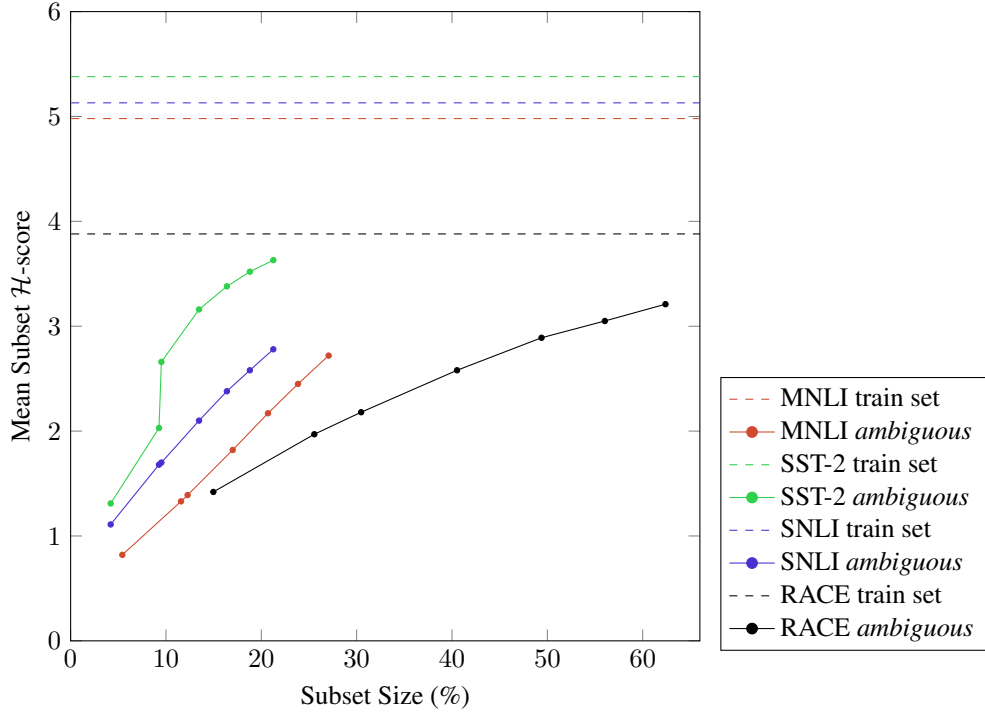


Figure 13: Mean $\mathcal{H}$ -score of training subsets along with the training set for MNLI, SST-2, and SNLI tasks, based on RoBERTa_{LARGE}

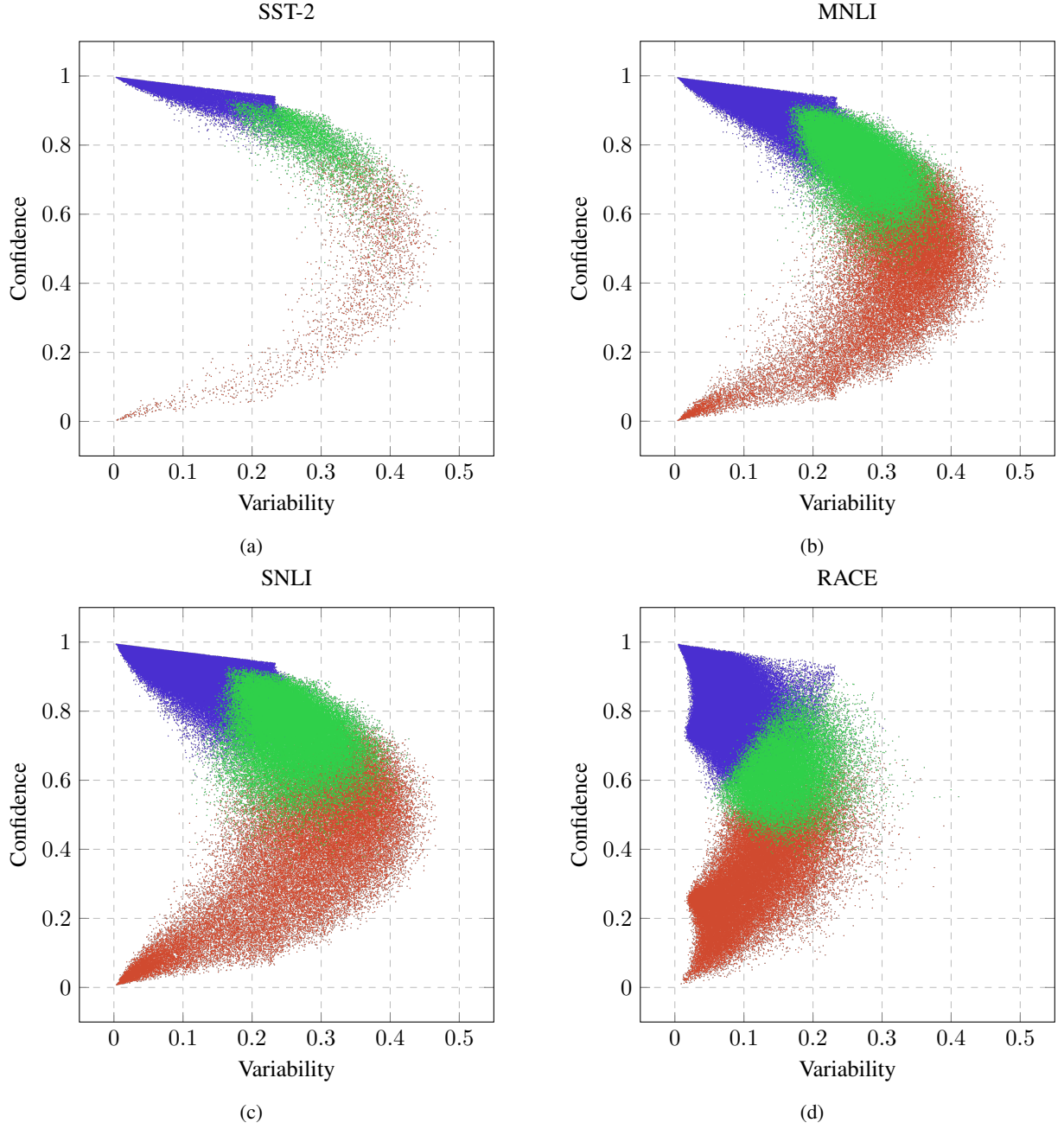


Figure 14: Data map for data points of the training set of various tasks, based on OPT_{350M}. Red dots represent data points with $\mathcal{H}_i \in \{0, 1\}$, green dots represent data points with $\mathcal{H}_i \in \{2, 3, 4\}$, and blue dots represent data points with $\mathcal{H}_i \in \{5, 6\}$.

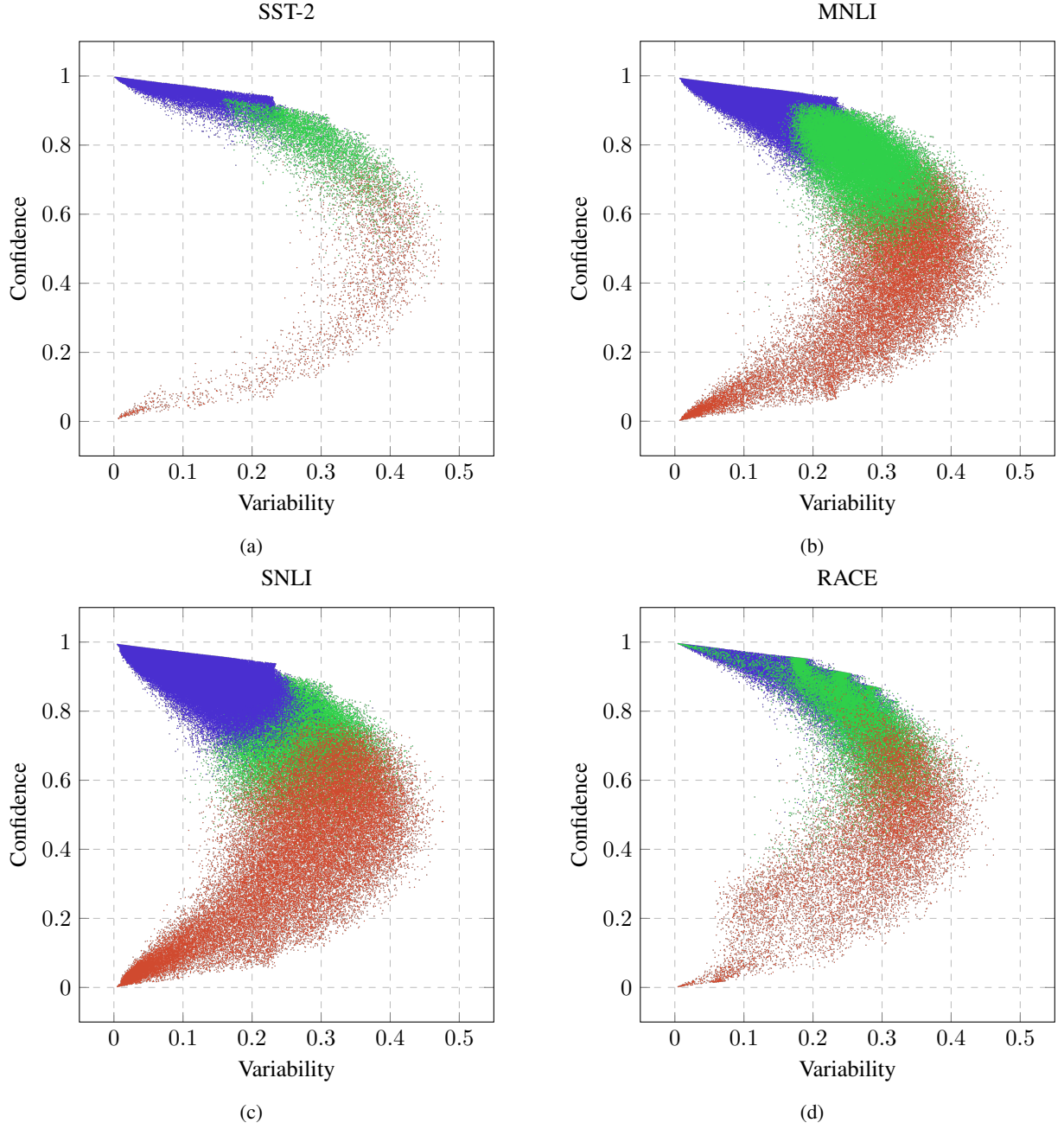


Figure 15: Data map for data points of the training set of various tasks, based on RoBERTa_{LARGE}. Red dots represent data points with $\mathcal{H}_i \in \{0, 1\}$, green dots represent data points with $\mathcal{H}_i \in \{2, 3, 4\}$, and blue dots represent data points with $\mathcal{H}_i \in \{5, 6\}$.

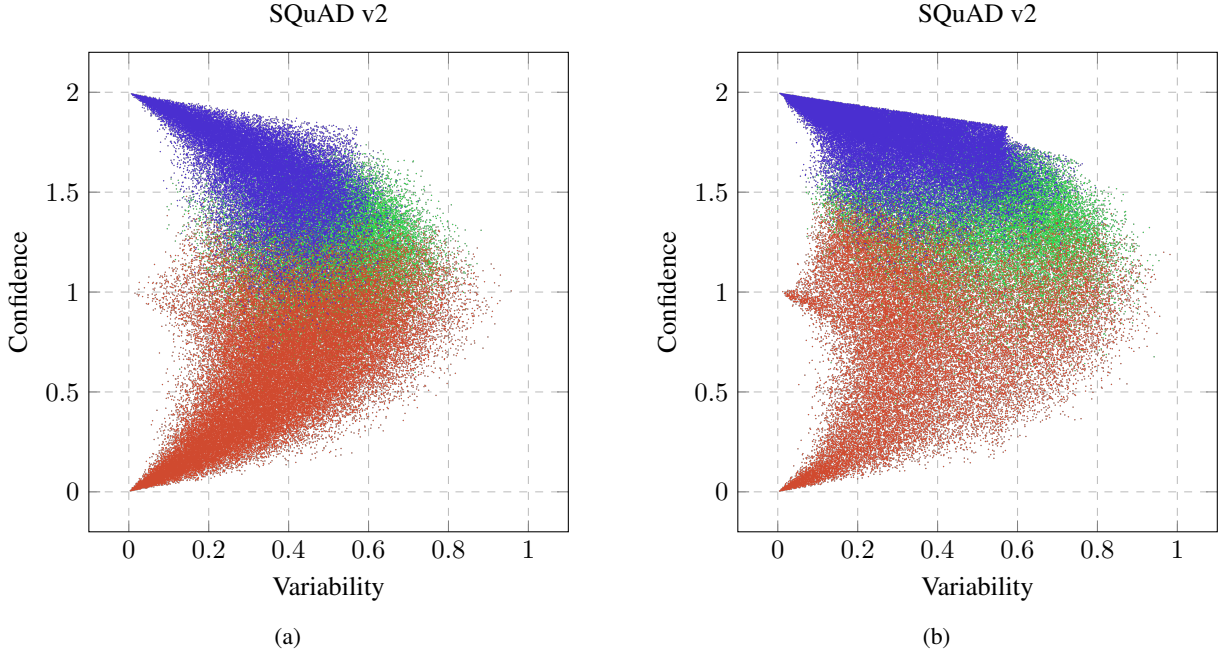


Figure 16: Data map for data points of the training set of SQuAD v2 task, based on OPT_{350M} (left) and RoBERTa_{LARGE} (right). Red dots represent data points with $\mathcal{H}_i \in \{0, 1\}$, green dots represent data points with $\mathcal{H}_i \in \{2, 3, 4\}$, and blue dots represent data points with $\mathcal{H}_i \in \{5, 6\}$.

A.6 COMPARISON WITH FORGETTING SCORE

We introduced $\mathcal{F}$ -score in Section 4.3 to study the main difference between our $\mathcal{H}$ -score and the forgetting score [Toneva et al. \(2019\)](#). Figure 17 shows the distribution of both scores side by side. Across all tasks, $\mathcal{F}$ -score reduces the number of data points with the score smaller than 6. This demonstrates that many of the data points that were learned at a later epoch in one of the runs, were correctly classified in all epochs in other runs. While under $\mathcal{H}$ -score these data points receive a reward of 1 only in the latter type of runs, $\mathcal{F}$ -score rewards all of them and thus leads to a score of 6.

We use $\mathcal{F}$ -score to create subsets similar to the experiments in Section 4.1. Tables 6 and 7 compare the evaluation performance of RoBERTa_{LARGE} and OPT_{350M} on subsets made using $\mathcal{F}$ -score with subsets made based on $\mathcal{H}$ -score. The experimental results demonstrate that using $\mathcal{F}$ -score results in smaller subsets and also significant drops in the evaluation performance.

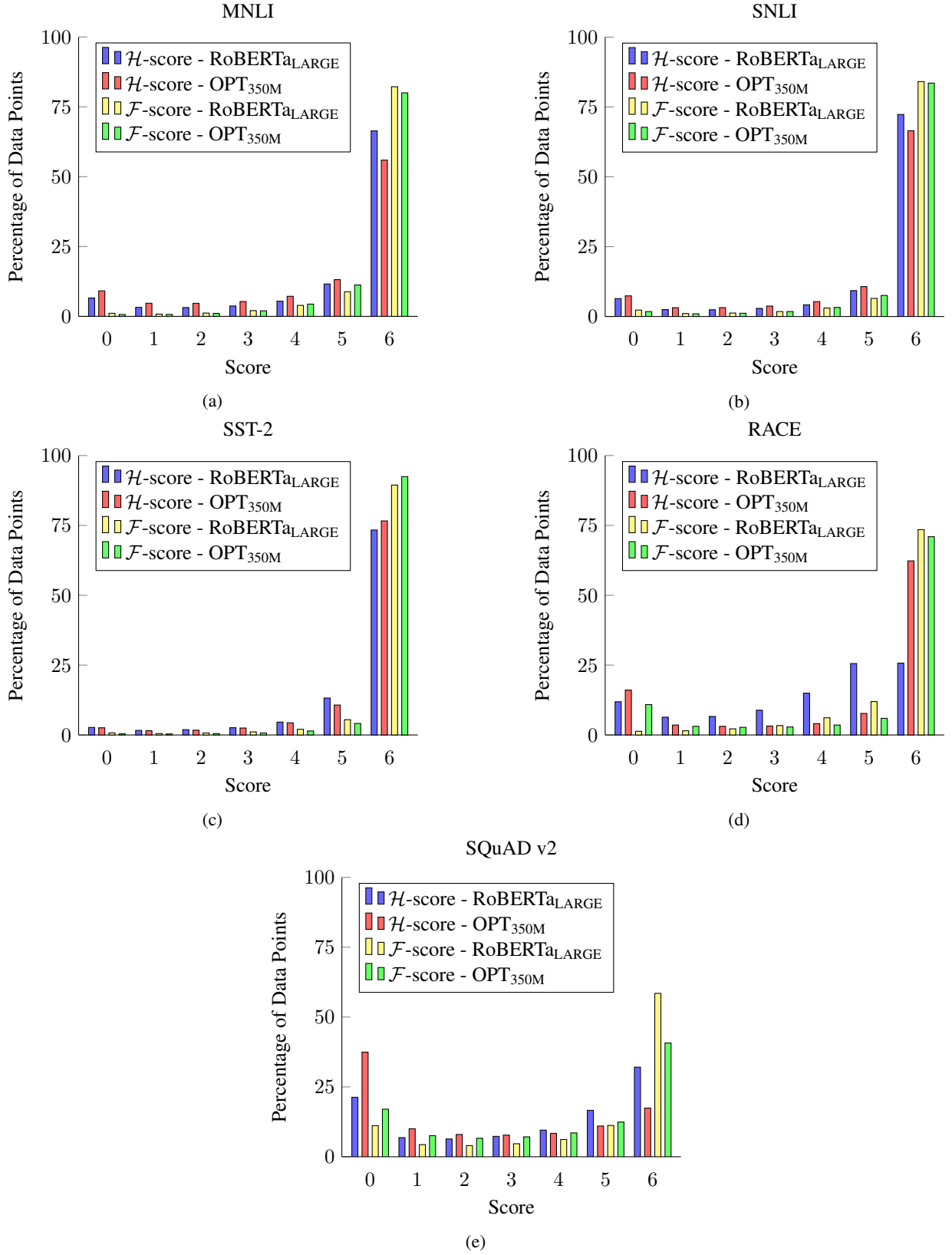


Figure 17: Comparison between the distribution of the $\mathcal{H}$ -score and $\mathcal{F}$ -score. Scores are reported for the training set of the MNLI, SNLI, SST-2, RACE, and SQuAD v2 tasks, based on RoBERTa_{LARGE} and OPT_{350M} models.

MNLI Matched (Accuracy)								
		$\mathcal{D}_{\{4\}}$	$\mathcal{D}_{\{5\}}$	$\mathcal{D}_{\{2,3,4\}}$	$\mathcal{D}_{\{4,5\}}$	$\mathcal{D}_{\{3,4,5\}}$	$\mathcal{D}_{\{2,3,4,5\}}$	$\mathcal{D}_{\{1,2,3,4,5\}}$
RoBERTa Large	$\mathcal{H}$ -score	88.68 _{0.50} (5.42%)	89.48 _{0.09} (11.58%)	89.91 _{0.24} (12.27%)	90.06 _{0.13} (17.0%)	90.23 _{0.11} (20.7%)	90.36 _{0.09} (23.85%)	90.57 _{0.12} (27.06%)
	$\mathcal{F}$ -score	79.77 _{0.47} (3.92%)	88.34 _{0.14} (8.8%)	43.53 _{11.39} (7.14%)	87.54 _{1.54} (12.72%)	87.86 _{0.43} (14.77%)	50.49 _{32.31} (15.94%)	76.45 _{12.16} (16.73%)
OPT 350M	$\mathcal{H}$ -score	80.87 _{0.39} (7.18%)	81.87 _{0.14} (13.14%)	81.99 _{0.44} (17.14%)	82.37 _{0.19} (20.32%)	83.23 _{0.14} (25.62%)	83.38 _{0.11} (30.28%)	83.53 _{0.37} (34.97%)
	$\mathcal{F}$ -score	60.09 _{2.01} (4.38%)	77.65 _{0.26} (11.22%)	51.27 _{1.81} (7.43%)	76.20 _{0.29} (15.61%)	75.15 _{0.67} (17.59%)	73.30 _{0.31} (18.65%)	69.95 _{1.59} (19.34%)
MNLI Mismatched (Accuracy)								
		$\mathcal{D}_{\{4\}}$	$\mathcal{D}_{\{5\}}$	$\mathcal{D}_{\{2,3,4\}}$	$\mathcal{D}_{\{4,5\}}$	$\mathcal{D}_{\{3,4,5\}}$	$\mathcal{D}_{\{2,3,4,5\}}$	$\mathcal{D}_{\{1,2,3,4,5\}}$
RoBERTa Large	$\mathcal{H}$ -score	88.79 _{0.35} (5.42%)	89.26 _{0.25} (11.58%)	89.61 _{0.18} (12.27%)	89.81 _{0.14} (17.0%)	90.05 _{0.04} (20.7%)	90.12 _{0.31} (23.85%)	90.22 _{0.05} (27.06%)
	$\mathcal{F}$ -score	80.19 _{0.47} (3.92%)	88.19 _{0.14} (8.8%)	43.95 _{11.39} (7.14%)	87.53 _{1.54} (12.72%)	87.93 _{0.43} (14.77%)	50.48 _{32.31} (15.94%)	77.50 _{12.16} (16.73%)
OPT 350M	$\mathcal{H}$ -score	81.83 _{0.35} (7.18%)	82.56 _{0.25} (13.14%)	82.85 _{0.18} (17.14%)	83.55 _{0.14} (20.32%)	83.70 _{0.04} (25.62%)	84.15 _{0.31} (30.28%)	84.29 _{0.05} (34.97%)
	$\mathcal{F}$ -score	61.85 _{1.63} (4.38%)	77.99 _{0.25} (11.22%)	52.52 _{2.13} (7.43%)	77.26 _{0.22} (15.61%)	75.68 _{0.53} (17.59%)	73.60 _{0.25} (18.65%)	70.91 _{1.20} (19.34%)
SNLI (Accuracy)								
		$\mathcal{D}_{\{4\}}$	$\mathcal{D}_{\{5\}}$	$\mathcal{D}_{\{2,3,4\}}$	$\mathcal{D}_{\{4,5\}}$	$\mathcal{D}_{\{3,4,5\}}$	$\mathcal{D}_{\{2,3,4,5\}}$	$\mathcal{D}_{\{1,2,3,4,5\}}$
RoBERTa Large	$\mathcal{H}$ -score	91.41 _{0.31} (4.2%)	91.84 _{0.15} (9.27%)	91.66 _{0.16} (9.52%)	92.13 _{0.22} (13.47%)	92.38 _{0.03} (16.39%)	92.41 _{0.03} (18.8%)	92.50 _{0.14} (21.26%)
	$\mathcal{F}$ -score	32.77 _{0.00} (3.04%)	71.70 _{33.71} (6.48%)	46.82 _{24.34} (6.08%)	32.77 _{0.00} (9.52%)	52.14 _{33.56} (11.3%)	42.51 _{16.86} (12.55%)	51.83 _{33.02} (13.62%)
OPT 350M	$\mathcal{H}$ -score	87.34 _{0.12} (5.34%)	88.44 _{0.03} (10.71%)	88.21 _{0.32} (12.26%)	89.13 _{0.13} (16.06%)	89.58 _{0.11} (19.79%)	89.87 _{0.14} (22.97%)	90.07 _{0.29} (26.13%)
	$\mathcal{F}$ -score	70.56 _{0.71} (3.25%)	84.32 _{0.07} (7.54%)	53.86 _{1.32} (6.19%)	84.07 _{0.21} (10.79%)	82.86 _{0.44} (12.57%)	80.86 _{0.17} (13.73%)	77.03 _{0.33} (14.7%)
SST-2 (Accuracy)								
		$\mathcal{D}_{\{4\}}$	$\mathcal{D}_{\{2,3,4\}}$	$\mathcal{D}_{\{5\}}$	$\mathcal{D}_{\{4,5\}}$	$\mathcal{D}_{\{3,4,5\}}$	$\mathcal{D}_{\{2,3,4,5\}}$	$\mathcal{D}_{\{1,2,3,4,5\}}$
RoBERTa Large	$\mathcal{H}$ -score	94.15 _{0.35} (4.6%)	94.65 _{0.46} (9.11%)	94.99 _{0.17} (13.21%)	95.45 _{0.24} (17.81%)	95.53 _{0.42} (20.41%)	95.87 _{0.23} (22.32%)	95.80 _{0.37} (23.95%)
	$\mathcal{F}$ -score	55.08 _{5.22} (2.04%)	50.92 _{0.00} (3.86%)	63.72 _{22.18} (5.47%)	93.31 _{0.82} (7.51%)	63.53 _{21.85} (8.61%)	54.89 _{6.88} (9.33%)	64.76 _{23.97} (9.84%)
OPT 350M	$\mathcal{H}$ -score	91.67 _{0.33} (4.37%)	90.98 _{0.35} (8.58%)	93.19 _{0.18} (10.71%)	93.81 _{0.12} (15.08%)	93.73 _{0.07} (17.57%)	94.19 _{0.29} (19.29%)	94.04 _{0.12} (20.82%)
	$\mathcal{F}$ -score	68.77 _{9.20} (1.43%)	49.16 _{12.11} (2.64%)	88.61 _{1.40} (4.12%)	88.34 _{0.70} (5.55%)	87.35 _{1.62} (6.26%)	84.82 _{1.34} (6.76%)	80.70 _{2.20} (7.13%)

Table 6: Evaluation accuracy of fine-tuning RoBERTa_{LARGE} and OPT_{350M} on subsets created with $\mathcal{H}$ -score and $\mathcal{F}$ -score. The size of each subset is provided in parenthesis as a percentage of the original dataset size. Each reported accuracy is the average of 3 runs with different initialization seeds. The standard deviation of the runs is also reported as the subscript.

RACE (Accuracy)							
		$\mathcal{D}_{\{4\}}$	$\mathcal{D}_{\{5\}}$	$\mathcal{D}_{\{2,3,4\}}$	$\mathcal{D}_{\{4,5\}}$	$\mathcal{D}_{\{3,4,5\}}$	$\mathcal{D}_{\{2,3,4,5\}}$
RoBERTa Large	$\mathcal{H}$ -score	77.79 _{0.39} (14.97)%	78.73 _{0.25} (25.56)%	80.56 _{0.73} (30.47)%	80.62 _{0.12} (40.53)%	81.36 _{0.43} (49.39)%	82.36 _{0.54} (56.03)%
	$\mathcal{F}$ -score	28.54 _{2.00} (6.15)%	23.75 _{1.22} (11.94)%	23.64 _{0.15} (11.73)%	40.22 _{29.11} (18.1)%	24.22 _{1.43} (21.47)%	24.94 _{0.08} (23.67)%
OPT 350M	$\mathcal{H}$ -score	76.64 _{0.42} (4.06)%	77.60 _{0.25} (7.71)%	74.59 _{0.20} (10.35)%	78.50 _{0.10} (11.77)%	78.85 _{0.06} (14.98)%	79.26 _{0.25} (18.06)%
	$\mathcal{F}$ -score	56.91 _{13.41} (3.55)%	74.07 _{1.66} (5.95)%	32.53 _{0.74} (9.14)%	71.39 _{2.08} (9.5)%	62.16 _{8.99} (12.36)%	48.22 _{10.33} (15.09)%
SQuAD v2 (Exact Match)							
		$\mathcal{D}_{\{4\}}$	$\mathcal{D}_{\{5\}}$	$\mathcal{D}_{\{2,3,4\}}$	$\mathcal{D}_{\{4,5\}}$	$\mathcal{D}_{\{3,4,5\}}$	$\mathcal{D}_{\{2,3,4,5\}}$
RoBERTa Large	$\mathcal{H}$ -score	82.13 _{0.20} (9.54)%	81.40 _{0.20} (16.61)%	84.03 _{0.34} (23.2)%	82.84 _{0.41} (26.15)%	83.80 _{0.22} (33.44)%	84.39 _{0.25} (39.81)%
	$\mathcal{F}$ -score	68.05 _{7.13} (6.18)%	81.76 _{0.28} (11.2)%	68.60 _{2.39} (14.85)%	82.74 _{0.52} (17.38)%	82.97 _{0.21} (22.05)%	82.78 _{0.49} (26.05)%
OPT 350M	$\mathcal{H}$ -score	59.75 _{0.34} (8.38)%	59.54 _{0.17} (10.99)%	61.00 _{0.14} (19.38)%	62.50 _{0.45} (24.10)%	61.77 _{0.21} (27.12)%	62.86 _{0.12} (35.09)%
	$\mathcal{F}$ -score	54.73 _{1.23} (8.53)%	59.59 _{0.57} (12.42)%	54.84 _{0.07} (22.29)%	61.32 _{0.31} (20.95)%	61.76 _{0.22} (28.08)%	61.21 _{0.60} (34.71)%
SQuAD v2 (F1 Score)							
		$\mathcal{D}_{\{4\}}$	$\mathcal{D}_{\{5\}}$	$\mathcal{D}_{\{2,3,4\}}$	$\mathcal{D}_{\{4,5\}}$	$\mathcal{D}_{\{3,4,5\}}$	$\mathcal{D}_{\{2,3,4,5\}}$
RoBERTa Large	$\mathcal{H}$ -score	84.94 _{0.07} (9.54)%	84.06 _{0.28} (16.61)%	86.62 _{0.26} (23.2)%	85.42 _{0.45} (26.15)%	86.42 _{0.30} (33.44)%	86.91 _{0.28} (39.81)%
	$\mathcal{F}$ -score	71.11 _{6.90} (6.18)%	84.59 _{0.31} (11.2)%	71.69 _{2.32} (14.85)%	85.40 _{0.48} (17.38)%	85.70 _{0.27} (22.05)%	85.53 _{0.51} (26.05)%
OPT 350M	$\mathcal{H}$ -score	62.19 _{0.19} (8.38)%	61.87 _{0.20} (10.99)%	63.47 _{0.10} (19.38)%	65.14 _{0.34} (24.10)%	64.30 _{0.31} (27.12)%	65.44 _{0.13} (35.09)%
	$\mathcal{F}$ -score	58.10 _{1.17} (8.53)%	62.56 _{0.45} (12.42)%	58.67 _{0.09} (22.29)%	64.31 _{0.29} (20.95)%	64.90 _{0.24} (28.08)%	64.56 _{0.64} (34.71)%

Table 7: Evaluation accuracy of fine-tuning RoBERTa_{LARGE} and OPT_{350M} on subsets created with $\mathcal{H}$ -score and $\mathcal{F}$ -score. The size of each subset is provided in parenthesis as a percentage of the original dataset size. Each reported accuracy is the average of 3 runs with different initialization seeds. The standard deviation of the runs is also reported as the subscript.

A.7 SENSITIVITY ANALYSIS ON S AND E HYPERPARAMETERS

We study the effect of number of runs, S , and number of epochs, E , on our pruned subsets. We run fine-tuning experiments on subsets created with $S \in \{2, 3, 4, 5, 6\}$ and $E \in \{1, 2, 3\}$. Note that our main experimental setup in Section 4.1 is done with $S = 6$ and $E = 3$ ³. Table 8 reports the analysis on S and Tables 9 and 10 report the analysis on E .

³For SQuAD v2, $E = 2$

Task	Model	$S = 2$	$S = 3$	$S = 4$	$S = 5$	$S = 6$
MNLI Matched Accuracy	RoBERTa _{LARGE}	90.04 _{0.24} (11.61%)	89.50 _{0.96} (17.62%)	83.52 _{11.75} (21.56%)	90.26 _{0.11} (24.61%)	90.57 _{0.12} (27.06%)
	OPT _{350M}	82.06 _{0.06} (15.5%)	82.82 _{0.17} (23.21%)	83.46 _{0.25} (28.26%)	83.56 _{0.32} (31.96%)	83.53 _{0.37} (34.97%)
MNLI Mismatched Accuracy	RoBERTa _{LARGE}	89.48 _{0.19} (11.61%)	89.22 _{0.96} (17.62%)	83.85 _{10.81} (21.56%)	90.16 _{0.25} (24.61%)	90.22 _{0.05} (27.06%)
	OPT _{350M}	82.90 _{0.20} (15.5%)	83.72 _{0.14} (23.21%)	84.14 _{0.11} (28.26%)	84.39 _{0.30} (31.96%)	84.29 _{0.05} (34.97%)
SNLI Accuracy	RoBERTa _{LARGE}	52.40 _{34.01} (9.23%)	92.33 _{0.03} (13.79%)	92.33 _{0.07} (16.91%)	92.46 _{0.06} (19.29%)	92.50 _{0.14} (21.26%)
	OPT _{350M}	88.72 _{0.08} (11.41%)	89.33 _{0.02} (17.14%)	89.58 _{0.16} (20.95%)	89.90 _{0.08} (23.83%)	90.07 _{0.29} (26.13%)
SST-2 Accuracy	RoBERTa _{LARGE}	95.07 _{0.12} (9.91%)	95.22 _{0.17} (14.7%)	95.80 _{0.18} (17.98%)	95.91 _{0.13} (20.32%)	95.80 _{0.37} (23.95%)
	OPT _{350M}	92.66 _{0.41} (9.02%)	93.16 _{0.29} (13.33%)	92.85 _{0.95} (16.43%)	92.97 _{0.59} (18.83%)	94.04 _{0.12} (20.82%)
RACE Accuracy	RoBERTa _{LARGE}	61.91 _{31.87} (28.03%)	81.77 _{0.23} (41.12%)	82.29 _{0.18} (50.33%)	82.89 _{0.40} (56.97%)	82.95 _{0.13} (62.39%)
	OPT _{350M}	76.16 _{0.38} (9.49%)	78.55 _{0.38} (14.05%)	79.36 _{0.11} (17.31%)	79.35 _{0.33} (19.68%)	79.68 _{0.09} (21.62%)
SQuAD v2 Exact Match	RoBERTa _{LARGE}	83.64 _{0.32} (19.44%)	84.24 _{0.38} (29.15%)	84.88 _{0.22} (35.67%)	84.92 _{0.31} (43.24%)	85.09 _{0.14} (46.66%)
	OPT _{350M}	62.15 _{0.17} (20.57%)	62.97 _{0.15} (30.74%)	63.23 _{0.24} (37.12%)	63.61 _{0.18} (41.63%)	63.72 _{0.38} (45.13%)
SQuAD v2 F1 Score	RoBERTa _{LARGE}	86.35 _{0.34} (19.44%)	86.98 _{0.36} (29.15%)	87.43 _{0.20} (35.67%)	87.53 _{0.29} (43.24%)	87.70 _{0.14} (46.66%)
	OPT _{350M}	64.80 _{0.17} (20.57%)	65.61 _{0.18} (30.74%)	65.80 _{0.23} (37.12%)	66.25 _{0.22} (41.63%)	66.49 _{0.22} (45.13%)

Table 8: Evaluation accuracy of fine-tuning RoBERTa_{LARGE} and OPT_{350M} on the winning ticket subset for different values of S . The size of each subset is provided in parenthesis as a percentage of the original dataset size. Each reported accuracy is the average of 3 runs with different initialization seeds. The standard deviation of the runs is also reported as the subscript.

MNLI Matched (Accuracy)							
		$\mathcal{D}_{\{4\}}$	$\mathcal{D}_{\{5\}}$	$\mathcal{D}_{\{2,3,4\}}$	$\mathcal{D}_{\{4,5\}}$	$\mathcal{D}_{\{3,4,5\}}$	$\mathcal{D}_{\{2,3,4,5\}}$
RoBERTa Large	$E = 3$	88.68 _{0.50} (5.42%)	89.48 _{0.09} (11.58%)	89.91 _{0.24} (12.27%)	90.06 _{0.13} (17.0%)	90.23 _{0.11} (20.7%)	90.36 _{0.09} (23.85%)
	$E = 2$	88.58 _{0.40} (5.41%)	89.35 _{0.11} (11.64%)	89.91 _{0.09} (12.18%)	89.95 _{0.16} (17.04%)	90.06 _{0.19} (20.7%)	90.25 _{0.09} (23.81%)
	$E = 1$	69.11 _{32.30} (5.33%)	69.98 _{33.05} (11.8%)	89.42 _{0.20} (11.86%)	69.82 _{32.92} (17.18%)	90.03 _{0.22} (20.75%)	90.41 _{0.10} (23.66%)
OPT 350M	$E = 3$	80.87 _{0.39} (7.18%)	81.87 _{0.14} (13.14%)	81.99 _{0.44} (17.14%)	82.37 _{0.19} (20.32%)	83.23 _{0.14} (25.62%)	83.38 _{0.11} (30.28%)
	$E = 2$	80.98 _{0.17} (7.14%)	81.83 _{0.09} (13.13%)	81.75 _{0.52} (16.98%)	82.51 _{0.11} (20.27%)	83.17 _{0.13} (25.54%)	83.52 _{0.12} (30.11%)
	$E = 1$	80.36 _{0.27} (7.11%)	81.81 _{0.24} (13.28%)	79.86 _{0.36} (16.75%)	82.56 _{0.18} (20.39%)	83.13 _{0.18} (25.62%)	83.46 _{0.29} (30.03%)
MNLI Mismatched (Accuracy)							
		$\mathcal{D}_{\{4\}}$	$\mathcal{D}_{\{5\}}$	$\mathcal{D}_{\{2,3,4\}}$	$\mathcal{D}_{\{4,5\}}$	$\mathcal{D}_{\{3,4,5\}}$	$\mathcal{D}_{\{2,3,4,5\}}$
RoBERTa Large	$E = 3$	88.79 _{0.35} (5.42%)	89.26 _{0.25} (11.58%)	89.61 _{0.18} (12.27%)	89.81 _{0.14} (17.0%)	90.05 _{0.04} (20.7%)	90.12 _{0.31} (23.85%)
	$E = 2$	88.40 _{0.43} (5.41%)	89.20 _{0.12} (11.64%)	89.80 _{0.13} (12.18%)	89.57 _{0.05} (17.04%)	89.95 _{0.05} (20.7%)	90.15 _{0.06} (23.81%)
	$E = 1$	69.10 _{32.29} (5.33%)	70.04 _{33.10} (11.8%)	89.34 _{0.12} (11.86%)	69.70 _{32.82} (17.18%)	90.17 _{0.19} (20.75%)	90.35 _{0.12} (23.66%)
OPT 350M	$E = 3$	81.83 _{0.35} (7.18%)	82.56 _{0.25} (13.14%)	82.85 _{0.18} (17.14%)	83.55 _{0.14} (20.32%)	83.70 _{0.04} (25.62%)	84.15 _{0.31} (30.28%)
	$E = 2$	81.77 _{0.15} (7.14%)	82.69 _{0.28} (13.13%)	82.73 _{0.03} (16.98%)	83.45 _{0.09} (20.27%)	83.86 _{0.08} (25.54%)	84.22 _{0.35} (30.11%)
	$E = 1$	81.23 _{0.11} (7.11%)	82.65 _{0.20} (13.28%)	80.57 _{0.34} (16.75%)	83.48 _{0.02} (20.39%)	83.93 _{0.15} (25.62%)	84.23 _{0.27} (30.03%)

Table 9: Evaluation accuracy of fine-tuning RoBERTa_{LARGE} and OPT_{350M} on subsets acquired with different values of E . The size of each subset is provided in parenthesis as a percentage of the original dataset size. Each reported accuracy is the average of 3 runs with different initialization seeds. The standard deviation of the runs is also reported as the subscript.

SNLI (Accuracy)								
	$\mathcal{D}_{\{4\}}$	$\mathcal{D}_{\{5\}}$	$\mathcal{D}_{\{2,3,4\}}$	$\mathcal{D}_{\{4,5\}}$	$\mathcal{D}_{\{3,4,5\}}$	$\mathcal{D}_{\{2,3,4,5\}}$	$\mathcal{D}_{\{1,2,3,4,5\}}$	
RoBERTa Large	$E = 3$	91.41 _{0.31} (4.2%)	91.84 _{0.15} (9.27%)	91.66 _{0.16} (9.52%)	92.13 _{0.22} (13.47%)	92.38 _{0.03} (16.39%)	92.41 _{0.03} (18.8%)	92.50 _{0.14} (21.26%)
	$E = 2$	60.03 _{29.49} (4.19%)	52.42 _{34.04} (9.38%)	32.77 _{0.00} (9.48%)	92.08 _{0.15} (13.58%)	52.52 _{34.21} (16.48%)	52.62 _{34.38} (18.87%)	72.56 _{34.46} (21.27%)
	$E = 1$	71.77 _{33.78} (4.2%)	32.77 _{0.00} (9.29%)	32.77 _{0.00} (9.48%)	52.56 _{34.27} (13.68%)	32.77 _{0.00} (16.51%)	52.55 _{34.26} (18.76%)	52.58 _{34.31} (20.95%)
OPT 350M	$E = 3$	87.34 _{0.12} (5.34%)	88.44 _{0.03} (10.71%)	88.21 _{0.32} (12.26%)	89.13 _{0.13} (16.06%)	89.58 _{0.11} (19.79%)	89.87 _{0.14} (22.97%)	90.07 _{0.29} (26.13%)
	$E = 2$	87.23 _{0.23} (5.31%)	88.46 _{0.27} (10.75%)	87.63 _{0.72} (12.13%)	89.15 _{0.06} (16.05%)	89.57 _{0.15} (19.76%)	89.74 _{0.14} (22.88%)	89.76 _{0.25} (25.97%)
	$E = 1$	85.44 _{1.17} (5.29%)	88.33 _{0.05} (10.88%)	84.12 _{0.81} (11.95%)	88.99 _{0.10} (16.17%)	89.37 _{0.17} (19.78%)	89.59 _{0.17} (22.83%)	89.33 _{0.10} (25.77%)
SST-2 (Accuracy)								
	$\mathcal{D}_{\{4\}}$	$\mathcal{D}_{\{2,3,4\}}$	$\mathcal{D}_{\{5\}}$	$\mathcal{D}_{\{4,5\}}$	$\mathcal{D}_{\{3,4,5\}}$	$\mathcal{D}_{\{2,3,4,5\}}$	$\mathcal{D}_{\{1,2,3,4,5\}}$	
RoBERTa Large	$E = 3$	94.15 _{0.35} (4.6%)	94.65 _{0.46} (9.11%)	94.99 _{0.17} (13.21%)	95.45 _{0.24} (17.81%)	95.53 _{0.42} (20.41%)	95.87 _{0.23} (22.32%)	95.80 _{0.37} (23.95%)
	$E = 2$	63.45 _{24.90} (4.65%)	49.04 _{0.06} (9.07%)	64.45 _{26.62} (13.3%)	95.10 _{0.27} (17.95%)	80.24 _{26.98} (20.53%)	95.49 _{0.17} (22.37%)	95.87 _{0.46} (23.91%)
	$E = 1$	62.08 _{22.31} (4.44%)	49.08 _{0.00} (8.48%)	95.26 _{0.24} (13.49%)	80.43 _{27.15} (17.93%)	79.89 _{26.68} (20.4%)	79.13 _{26.08} (21.96%)	94.04 _{3.49} (23.25%)
OPT 350M	$E = 3$	91.67 _{0.33} (4.37%)	90.98 _{0.35} (8.58%)	93.19 _{0.18} (10.71%)	93.81 _{0.12} (15.08%)	93.73 _{0.07} (17.57%)	94.19 _{0.29} (19.29%)	94.04 _{0.12} (20.82%)
	$E = 2$	87.27 _{5.76} (4.4%)	88.91 _{3.15} (8.59%)	92.55 _{0.31} (10.75%)	93.27 _{0.48} (15.15%)	93.04 _{0.78} (17.62%)	93.50 _{0.57} (19.34%)	93.20 _{0.27} (20.85%)
	$E = 1$	90.48 _{1.43} (4.41%)	72.59 _{13.93} (8.43%)	93.08 _{0.29} (10.75%)	93.08 _{0.35} (15.16%)	93.85 _{0.24} (17.56%)	93.27 _{0.24} (19.18%)	92.85 _{0.35} (20.5%)

Table 10: Evaluation accuracy of fine-tuning RoBERTa_{LARGE} and OPT_{350M} on subsets acquired with different values of E . The size of each subset is provided in parenthesis as a percentage of the original dataset size. Each reported accuracy is the average of 3 runs with different initialization seeds. The standard deviation of the runs is also reported as the subscript.